Functional Test Case Descriptions

for

OpenBench

Tests of Release 1 against the Software Requirements Specification, version 0.4

Version 0.3, short edition

2026-11-07

Prepared by

Jai Nana Brian Lujan

Fotih Turopov Yerassyl Shnazbayev

CSDS 393 Software Engineering

Case Western Reserve University

Record of Changes

Name	Date	Reason for changes	Reviewed by	Version
J. Nana	2026-11-05	First complete draft: introduction, test environment and test world, approach, and test cases for all 47 functional requirements of SRS 0.4 with the traceability matrix.	B. Lujan, F. Turopov, Y. Shnazbayev	0.1
J. Nana	2026-11-07	Short edition added for submission: the High priority test cases and the Low priority cases of the Stretch requirements, which together cover every requirement; the Medium priority cases are kept in the full edition, and identifiers are the same in both.	B. Lujan, F. Turopov, Y. Shnazbayev	0.2
J. Nana	2026-11-07	Brought up to the build at commit `2d55477`: the Product Under Test now includes Member profiles and notes that the Priority mark and weekly recurrence are not built; the approach describes the `@Verifies` tags and the run report of `mvn verify`; the Automated by fields of TC-PRF-01, TC-PRF-02, TC-PRF-04, TC-DRF-04, TC-DRF-06, TC-DRF-08, TC-EXC-03, and TC-ADM-03 name the new automated tests.	B. Lujan, F. Turopov, Y. Shnazbayev	0.3

How to read this document. Section 1 names the product and the build under test, the scope, and the conventions of the test case template; Section 2 gives the environment, the test accounts, the test world every case starts from, the relative dates, and the reference Quotes; Section 3 gives the approach, the pass and fail criteria, and the special considerations, including how cases for features not yet built are treated; Section 4 holds the test cases, one subsection per feature of the SRS in the SRS's order, each ending with the numbers of the cases kept in the full edition only; Appendix A maps every requirement to its test cases, and Appendix B defines the terms. This short edition holds the High priority test cases and the Low priority cases of the Stretch requirements, which together cover every requirement. A longer edition that adds the Medium priority cases, for secondary rules and refusals, is kept in the project repository; identifiers are the same in both.

Contents

Record of Changes 1

1 Introduction 4
1.1 Purpose . . . 4
1.2 Product Under Test . . . 4
1.3 Scope . . . 4
1.4 References . . . 4
1.5 Conventions . . . 4

2 Test Environment 6
2.1 Software and Hardware . . . 6
2.2 Installation and Configuration . . . 6
2.3 Test Accounts . . . 6
2.4 Test World . . . 6
2.5 Dates and Times . . . 7
2.6 Reference Quotes . . . 7

3 Test Approach and Special Considerations 8

3.1 Approach . . . 8
3.2 Pass and Fail Criteria . . . 8
3.3 Entry and Exit Criteria . . . 8
3.4 Special Considerations . . . 8

4 Functional Test Cases **9**
4.1 Inventory and Resource Types . . . 9
TC-INV-01 Define a new Exclusive Resource Type without a code change . . . 9
TC-INV-04 Resource Owner adds, edits, and retires an Exclusive Resource . . . 10
TC-INV-05 Pool Capacity of a Metered type is the sum of its Resources' units . . . 10
TC-INV-06 A reduction or retirement below booked capacity is refused, naming the earliest hour . . . 11
TC-INV-08 Resource timeline shows free, on Hold, and allocated parts with their holders . . . 11
4.2 Member Profiles . . . 11
TC-PRF-01 A Member maintains their own skills from the skill list . . . 12
TC-PRF-02 Availability ranges decide whether a Member is available for a window . . . 12
TC-PRF-03 A Member with an overlapping Loan is not a Candidate, and a Loan leaves availability unchanged . . . 13
4.3 Suggestions . . . 13
TC-SUG-01 A Member creates a Suggestion that has no effect until submitted . . . 13
TC-SUG-02 The Manager accepts Suggestions as a new line and into an existing line . . . 14
4.4 Draft and Submit a Request . . . 14
TC-DRF-01 A Manager creates a Draft for their own Team . . . 14
TC-DRF-02 Another Team's Request is not found . . . 15
TC-DRF-03 Exclusive and Metered lines show their Quotes and the Draft its sums . . . 15
TC-DRF-05 Invalid lines are refused as they are added . . . 16
TC-DRF-06 Lines are Required by default and may be split to make a part Optional . . . 16
TC-DRF-07 Submission refused while the Required sum exceeds the Available Balance . . . 17
TC-DRF-09 Submission locks every Quote and records the submission time . . . 17
TC-DRF-10 Withdrawing from Draft, Waiting, or Pending releases everything held . . . 18
4.5 Matched Allocation . . . 18
TC-MAT-01 The Shortlist holds the qualified Members of other Teams . . . 19
TC-MAT-03 The Manager selects exactly the line's quantity; a short Shortlist leaves the Request Waiting . . . 19
TC-MAT-04 The approval request goes to the Candidate's Manager, who decides . . . 20
TC-MAT-06 A refusal asks for a replacement without restarting the Objection Window . . . 20
TC-MAT-07 A refusal with no Candidate left refuses a Required line and drops an Optional one . . . 21
TC-MAT-08 A silent Manager's approval escalates to the Capacity Planner . . . 21
TC-MAT-10 An approved Candidate becomes a Loan and the lending Team earns . . . 22
4.6 Exclusive Allocation . . . 22
TC-EXC-01 An instance is never given to two overlapping windows . . . 22
TC-EXC-04 Waitlist order: Bumped first, then Priority, then submission time . . . 23
TC-EXC-05 Freed capacity goes to the first waiting line whose window fits . . . 23
TC-EXC-07 A Priority line Bumps the latest-submitted blocking standard line . . . 24
TC-EXC-08 A Bumped line is refunded in full, heads its waitlist, and returns its Request to Waiting 24
TC-EXC-09 No Bump inside the bump cutoff; the Priority line waits ahead of standard lines . . . 25
4.7 Metered Allocation . . . 25
TC-MET-01 A Metered line asks for units of a type and is all or nothing . . . 25
TC-MET-02 Pool Capacity is checked for every hour of the window . . . 26
TC-MET-03 A Team's Quota is checked for every hour; other Teams' units do not count . . . 26
TC-MET-05 [Stretch] The Capacity Planner defines service tiers for a Metered type . . . 27
TC-MET-06 [Stretch] A higher-tier Metered line Bumps a lower-tier one . . . 27
TC-MET-07 [Stretch] The Resource Owner records the units consumed, with no Ledger effect . . . 27
4.8 Request Lifecycle and Holds . . . 27
TC-LIF-01 A Request without Matched lines goes from Submitted straight to Allocated . . . 28
TC-LIF-02 If one Required line cannot be satisfied, nothing is held, allocated, or charged . . . 28
TC-LIF-03 A Request that needs an approval is Pending with Holds and held credits . . . 29

TC-LIF-04 A Hold blocks other Requests exactly as an allocation does . . . 29
TC-LIF-05 Held credits stop concurrent Pending Requests from exceeding the Balance . . . 29
TC-LIF-06 The Objection Window starts at submission and lasts two working days . . . 30
TC-LIF-07 On lapse, every Hold is released and no Hold is placed afterwards . . . 30
TC-LIF-08 When the last approval arrives, everything commits in one step . . . 31
TC-LIF-09 At allocation, Optional lines that cannot be met are dropped with their reason . . . 31
TC-LIF-10 An Optional line's approval never holds a Request in Pending . . . 32
TC-LIF-11 A Waiting Request is allocated only when every Required line fits at once . . . 32
TC-LIF-13 Two Managers submitting for the last room at once: exactly one is allocated . . . 33
4.9 Release and Refund . . . 33
TC-REL-01 Release a whole line seven or more days ahead for a 75% Refund . . . 33
TC-REL-02 Partial Release shortens the window, reduces units, or removes occurrences . . . 34
TC-REL-03 The Refund fraction follows the lead time: 25% under seven days, 0% once started . . . 34
TC-REL-05 Ending a Loan early corrects the lending Team's Earning . . . 35
4.10 Ledger and Pricing . . . 35
TC-LED-01 Balance, held credits, and Available Balance . . . 35
TC-LED-02 The Statement lists the Period's Transactions, and each Team sees only its own . . . 36
TC-LED-03 Transactions are never updated or deleted; a correction is a new Transaction . . . 36
TC-LED-05 At the Period boundary, granted credits expire, earned credits stay, and the new Grant posts . . . 37
TC-LED-06 Quotes follow the price formula with its bands and the priority premium . . . 37
TC-LED-07 An Earning counts only Monday-to-Friday days of the Loan . . . 38
TC-LED-08 [Stretch] A demand surcharge raises the Quote with the share of capacity taken and is locked . . . 38
4.11 Showback Dashboard . . . 38
TC-DSH-01 Utilization, idle capacity, and waiting lines per Resource Type . . . 39
TC-DSH-02 Credits and Loans per Team and Period . . . 39
TC-DSH-04 [Stretch] Requested versus used units per Team and Metered type . . . 40
4.12 Administration . . . 40
TC-ADM-01 Create a Member who can sign in; a deactivated Member cannot . . . 40
TC-ADM-02 Teams, moves between Teams, and roles keep exactly one Manager per Team . . . 41
TC-ADM-04 Roles are enforced by the server, not only by the client . . . 41
TC-ADM-05 The Capacity Planner sets each policy value as data, and each change is recorded . . . 42
TC-ADM-06 A changed setting affects only what comes after the change . . . 42

A Traceability Matrix **43**

B Glossary **45**

1 Introduction

1.1 Purpose

This document describes the functional test cases for Release 1 of OpenBench. Each test case checks one behavior that a functional requirement of the Software Requirements Specification (SRS) states, from the point of view of the person using the system: what is set up beforehand, what the tester does, what data is entered, and what must be observed for the case to pass. The document is used by the team to run the tests and record their results, and by a reviewer to check that every requirement is tested.

1.2 Product Under Test

OpenBench is an internal resource marketplace. A Team's Manager posts one Request that can combine people (Matched lines, lent by other Teams), Exclusive resources such as meeting rooms and instruments, and Metered capacity such as GPU units. The system allocates each line under the policy of its Allocation Class, places Holds while lending Managers approve, keeps Requests that cannot be met yet on waitlists, and settles everything in credits: Teams receive a Grant each Period, pay a Charge for Exclusive and Metered capacity, and receive an Earning when they lend a Member.

The product is a single web application: a Spring Boot server with a PostgreSQL database and a React client in the browser, which talks to the server only through a REST API. The build under test is the `openbench` repository; the requirements are those of SRS version 0.4, and the product follows Software Design Document (SDD) version 1.1.

The product is being built in increments while this document is written. At commit `2d55477` (2026-11-07) the build implements Inventory and Resource Types, sign-in and roles, Member profiles with the skill list and availability, the Ledger with Statements, Draft Requests with Quotes, submission and withdrawal, and Exclusive and Metered allocation with waitlist entry; the Priority mark and the weekly recurrence of Exclusive lines are not built yet. Matched allocation, Pending Requests with Holds across approvals, the Objection Window, Release and Refund, Bumps, Suggestions, the dashboard, notifications, and the Administrator and Capacity Planner screens are designed but not yet built. This document specifies the test cases for the whole of Release 1 regardless; Section 3.4 explains how the cases for features not yet built are treated.

1.3 Scope

In scope. All 47 functional requirements of the SRS short edition, Section 4: the 41 Release 1 requirements and the 6 marked Stretch, which have their own test cases marked as such. Appendix A lists every requirement with the test cases that cover it.

Out of scope. The non-functional requirements of SRS Section 6 (performance, availability, usability) are not tested here, except where a functional test case also exercises a security rule such as role checks or Team scoping. Unit tests, the architecture test, and the client's component tests are described in the SDD's testing strategy and are referred to here only where an automated test runs a test case.

1.4 References

- OpenBench Software Requirements Specification, version 0.4, 2026-11-07 (short edition: the requirements tested here).
- OpenBench Software Design Document, version 1.1, 2026-11-07 (screens, API conventions, error codes, testing strategy).
- Architecture decision records ADR-0001 to ADR-0007, in particular ADR-0001 (Holds bounded by the Objection Window) and ADR-0006 (allocation correctness in the database).
- The `openbench` repository `README.md`: building, running with the demo data, and the demo logins.

1.5 Conventions

Each test case has an identifier **TC-***area***-***nn*, where the area is the SRS feature it belongs to (Table 1), and is written in the template of Table 2. Requirement identifiers such as **Draft.Quote.Line** are those of the SRS and link to the requirement's row of the traceability matrix. Capitalized words such as Request, Hold, and Quote have the meaning given in the glossary, Appendix B. Credits are shown with two decimals. Dates and times use the names of Section 2.5, such as "D 10:00–12:00", and are in the company time zone; a window includes its start hour and excludes its end hour. Error codes such as `INSUFFICIENT_BALANCE` are the `code` field of the API's problem detail; the client shows the problem's message with it.

Table 1: Test case areas, in the order of the SRS features.

Code	Section	SRS feature
INV	4.1	Inventory and Resource Types
PRF	4.2	Member Profiles
SUG	4.3	Suggestions
DRF	4.4	Draft and Submit a Request
MAT	4.5	Matched Allocation
EXC	4.6	Exclusive Allocation
MET	4.7	Metered Allocation
LIF	4.8	Request Lifecycle and Holds
REL	4.9	Release and Refund
LED	4.10	Ledger and Pricing
DSH	4.11	Showback Dashboard
ADM	4.12	Administration

Table 2: Fields of the test case template.

Field	Content
Requirements	The SRS identifiers the test case verifies.
Actor	The login, from Table 3, that performs the steps; a step performed by someone else names them.
Priority	High for the main behavior of a Release 1 requirement, Medium for a secondary rule or refusal, Low for a Stretch requirement.
Preconditions	The state the system must be in before step 1, beyond the test world of Section 2.4.
Test data	The lines, values, and amounts the steps use, named so the steps can refer to them.
Steps	Numbered rows: the Action the tester takes, its Input, and the Expected result that decides pass or fail.
Pass criteria	What must hold at the end, beyond the expected result of each step.
Automated by	The automated test that also runs the case, as test class and display name, or “None yet”.

2 Test Environment

2.1 Software and Hardware

Component	Version and notes
Server	Java 25, Spring Boot 3.5, one process serving the API under `/api` and the web client.
Database	PostgreSQL 16 (image `postgres:16-alpine`) in Docker, started from `compose.yaml`; the schema is created by the Flyway migrations of the build.
Web client	React 19 built with Vite 7, packed into the server's jar. Tested in a current desktop version of Chrome or Firefox at a window width of at least 1280 pixels.
API client	For the steps that call the API directly: `curl` with a cookie jar, or the browsable API page at `/api/docs/ui` while signed in.
Automated tests	Maven 3.9 (`mvn verify`), Docker for the Testcontainers PostgreSQL 16 database, Node 24 for the client tests.
Machines	The team's laptops (macOS and Linux) with Docker running; no special hardware.

2.2 Installation and Configuration

1. Check out the build under test (Section 1.2) and start Docker.
2. Start the application with the demo data: `mvn spring-boot:run -Dspring-boot.run.profiles=demo`. The server starts the database container, migrates it, and loads the seed data.
3. Open `http://localhost:8080` and sign in with a login of Table 3.

The company time zone is `America/New_York` (setting `OPENBENCH_ZONE`); every time in this document is in that zone. **Reset.** To return to the test world, stop the application, run `docker compose down` (which removes the database container and its data), and start the application again. The database is reset at the start of each subsection of Section 4, and again before any test case whose preconditions say "fresh test world". The seed posts the Grants of the Period in which the database is created, and the build under test has no Period rollover yet, so a database created before a Period ended is reset before the next test case runs.

2.3 Test Accounts

Every seeded Member's password is `demo`; every Member also holds the role Member.

Table 3: Test accounts.

Login	Name	Team	Roles besides Member
`alice`	Alice Chen	Platform	Manager
`ben`	Ben Ortiz	Platform	
`bob`	Bob Patel	Research	Manager
`rita`	Rita Gomez	Research	
`oscar`	Oscar Reed	Operations	Manager
`carol`	Carol Davis	Operations	Administrator
`dave`	Dave Kim	Operations	Resource Owner
`erin`	Erin Walsh	Operations	Capacity Planner

2.4 Test World

Every test case starts from the following data unless its preconditions say otherwise. Items 1 to 4 are loaded by the demo seed; item 5 is entered through the Administrator's skill list and each Member's profile before the first test case of Sections 4.2, 4.5, and 4.8 (the automated tests create it in their setup).

1. **Resource Types and Resources** (all owned by `dave`):
   - Meeting room (Exclusive), attributes `seats` (number) and `wing` (North or South): Room A (8 seats, North) and Room B (12 seats, South).
   - Confocal microscope (Exclusive), attribute `lab` (text): Microscope 1 (Imaging lab).
   - GPU (Metered), attribute `model` (A100 or H100): gpu-node-a (8 units, H100) and gpu-node-b (8 units, A100); Pool Capacity 16 units.
   - Person (Matched), the built-in type.
2. **Prices.** Unit Price per instance-hour or unit-hour: Meeting room 10.00, Confocal microscope 40.00, GPU 5.00.

Price Bands for each type: Monday to Friday 09:00–17:00 ×1.5, Saturday and Sunday all day ×0.5, every other hour ×1.0. Priority premium ×2. Day Rate 400.00 per person-day. Refund Schedule for each type: 75% at 168 hours (seven days) or more before the released part starts, 25% below that, 0% once it has started. Bump cutoff 24 hours for both Exclusive types.

3. **Accounts.** Each Team's Grant for the current Period, the calendar month in which the database was created, has been posted at the start of that Period and nothing else: Available Balance Platform 5000.00, Research 600.00, Operations 1000.00; no held or earned credits.
4. **Other settings.** Quota: Research may hold at most 8 GPU units concurrently; no other Quota. Objection Window two working days. Period one calendar month; Periods exist for ten years from the month in which the database was created.
5. **Matched data.** Skill list: Statistics, Java, Microscopy. Skills and availability (each available from the day of execution to D+60):
   - Ben Ortiz (Platform): Java.
   - Rita Gomez (Research): Statistics, Microscopy.
   - Dave Kim (Operations): Statistics.
   - Every other Member: no skills, no availability.

   No Loans exist. For a Platform line asking for Statistics the Shortlist is therefore Dave Kim, then Rita Gomez (no person-days lent, ordered by name); for a Research line it is Dave Kim alone.

2.5 Dates and Times

Windows must start in the future, so test cases name days relative to the day the case is run:

Name	Meaning
D	The first Wednesday at least 14 days after the day of execution. Every Release from a window on D has a lead time of seven days or more, and every window on D starts outside the bump cutoff.
D+n	The day n days after D: D+1 is a Thursday, D+3 a Saturday, D+7 the next Wednesday.
E	A weekday two to six days after the day of execution: its windows have a lead time under seven days.
H	The first whole hour at least two hours after the start of the test case: a window starting at H starts inside the 24-hour bump cutoff.

Time control. The server reads the time only from one injected clock. The automated integration tests replace it with a clock they move by hand, so they can lapse an Objection Window, start a line's window, or cross a Period boundary in one run. The running demo server uses the real clock, so a test case with a step marked **[clock]** is run either by its automated test or manually by waiting for the stated time to pass (for example, two working days for an Objection Window).

2.6 Reference Quotes

The Quotes that the test cases use, computed from the prices of Section 2.4 as quantity or units × Unit Price × band multiplier, summed over the hours:

Table 4: Reference Quotes.

Ref.	Line	Quote
Q1	Meeting room, quantity 1, D 10:00–12:00: $2 \times 10.00 \times 1.5$	30.00
Q2	Meeting room, quantity 1, D 16:00–18:00: $10.00 \times 1.5 + 10.00 \times 1.0$	25.00
Q3	Meeting room, quantity 1, D+3 (Saturday) 10:00–12:00: $2 \times 10.00 \times 0.5$	10.00
Q4	Meeting room, quantity 1, D 10:00–12:00, Priority: 30.00×2	60.00
Q5	Meeting room, quantity 1, D 10:00–12:00 weekly on D, D+7, and D+14: 3×30.00	90.00
Q6	Meeting room, quantity 1, D 09:00–17:00: $8 \times 10.00 \times 1.5$	120.00
Q7	GPU, 4 units, D 10:00–12:00: $4 \times 2 \times 5.00 \times 1.5$	60.00
Q8	GPU, 4 units, D 16:00–18:00: $4 \times 5.00 \times 1.5 + 4 \times 5.00 \times 1.0$	50.00
Q9	Confocal microscope, quantity 1, D 09:00–17:00: $8 \times 40.00 \times 1.5$	480.00
Q10	Confocal microscope, quantity 1, D+1 09:00–12:00: $3 \times 40.00 \times 1.5$	180.00
Q11	GPU, 6 units, D 10:00–12:00: $6 \times 2 \times 5.00 \times 1.5$	90.00
EA	Earning for a Loan of days D to D+1 (Wednesday and Thursday): 2×400.00	800.00

3 Test Approach and Special Considerations

3.1 Approach

The test cases are black-box tests derived from the requirements. For each requirement there is a case for the behavior it specifies and a case for each refusal, limit, or boundary it states; the cases for secondary rules and refusals have Medium priority, and the short edition of this document leaves them out. A case is run by hand through the web client, as the user named in its Actor field, so it shows what that user sees. Where the client has no screen for a check, such as calling an endpoint without the needed role, the step calls the API and says so. Where an automated test runs the same case, the Automated by field names it: those tests run on PostgreSQL 16 in Testcontainers with the production migrations through `mvn verify`, and each is tagged with the keys of the cases it runs (`@Verifies`), so that `mvn verify` also writes a run report, `target/test-cases/report.md`, with every case's outcome in that build: Passed, Passed in part, Failed, or Not automated. The cases are run in the order of Section 4, but each case's preconditions are complete, so a single case can be run alone after a reset.

3.2 Pass and Fail Criteria

A test case **passes** when the expected result of every step is observed and its pass criteria hold. "Observed" means seen on the screen the step names, or, for a step that calls the API, in the response's status and body. The Request detail's history, the Request list, the Inventory, and the Team's Statement are the places where state, Holds, and Transactions are checked. A test case **fails** when any expected result is not observed, or when the system changes anything the case says must not change, such as a Statement that gains a Transaction after a refused submission. A case that cannot be run because an earlier case or a precondition failed is **blocked**, not failed. Each run records the date, the build, the tester, the outcome of each case, and for a failure the step, what was observed, and the defect raised in the team's issue tracker.

3.3 Entry and Exit Criteria

Entry. The build under test compiles, `mvn verify` passes, and the application starts with the demo profile and shows the test world of Section 2.4. **Exit.** Every test case of a built feature has been run on the build under test and its result recorded; every High priority case passes; every failing case has a defect raised and assigned.

3.4 Special Considerations

- **Features not yet built.** The cases for features that the build under test does not implement yet (Section 1.2) are written against the SRS and the screens of the SDD, and their Automated by field reads "None yet". Until the feature is built they are recorded as *not run*, never as failed, and they are run when the feature's increment is delivered. A case's steps do not change when the feature is built, unless a screen named in the case turns out differently, in which case the step is reworded and the change entered in the Record of Changes. A case that is built in part, such as one whose last step checks a notification or that adds a Person line beside Exclusive and Metered ones, is run up to the first step that needs an unbuilt feature; the steps from there are recorded as not run, and the case counts as passed only once every step has run and passed.
- **Stretch requirements.** The six Stretch requirements have Low priority cases. They are run only if the feature is built for Release 1; otherwise they are recorded as not in the release.
- **Time.** Cases whose steps are marked [clock] depend on time passing, as described in Section 2.5. Windows must start in the future, so every case uses the relative days of that section.
- **Concurrency.** Two Managers racing for the last room or the last GPU units cannot be timed reliably by hand. Those cases are run by their automated tests, which start both submissions at once; a manual run checks only that the outcome is one allocation and one waiting line.
- **Roles and scoping.** The server enforces roles and Team scope, not the client. A Request, Account, or Statement of another Team answers "not found" (`NOT_FOUND`), and an action without its role answers `FORBIDDEN`. Cases that check this call the API directly, because the client never offers the forbidden action.
- **Notifications.** Notifications are in-app only. A step that expects a notification checks the recipient's notification list after signing in as that Member.
- **Data.** The test world uses only invented names and the `openbench.example` mail domain; no real personal data is used.

4 Functional Test Cases

One subsection per feature of SRS Section 4, in the same order. Each opens with what the feature's test cases establish; every test case uses the template of Section 1.5, starts from the test world of Section 2.4 after a reset, and uses the dates of Section 2.5 and the Quotes of Table 4. Requests named in preconditions, such as "Request A1", are created by the Actor named there through the Draft editor and submitted before step 1.

This edition holds the 63 High priority test cases and the 5 Low priority cases of the Stretch requirements; together they cover every requirement, as Appendix A shows. The 23 Medium priority cases, which check secondary rules and refusals, are kept in the full edition. Test cases keep the numbers they have there, so the numbers below skip those cases, and each subsection ends with a list of the ones it leaves out.

4.1 Inventory and Resource Types

These cases establish that Resource Types are data an Administrator defines without a code change; that a Resource Owner adds, changes, and retires Resources, and cannot cut capacity below what is already booked; and that any Member can see which Resources are free and who holds the others.

TC-INV-01	**Define a new Exclusive Resource Type without a code change**		
Requirements	**Inventory.Type.Define**		
Actor	`carol` (Administrator); `dave` and `alice` where named		
Priority	High		
Preconditions	Test world. No type named "3D printer" exists.		
Test data	Type T1: name "3D printer", class Exclusive, attributes `material` (list: PLA, ABS) and `volume` (number).		
Step	**Action**	**Input**	**Expected result**
1	Open Resource Types and choose to define a type.	T1	The form offers the classes Matched, Exclusive, and Metered and the attribute kinds text, number, date, and list.
2	Save the type.	None	T1 is listed at once with its class and both attributes. The server is not restarted or redeployed.
3	As `dave`, add a Resource of T1 from the Inventory.	Name "Printer 1", material PLA, volume 20	Printer 1 is listed in the Inventory under 3D printer.
4	As `alice`, create a Draft and open the Resource Type list of the line form.	None	3D printer is offered.
Pass criteria	The new type is usable end to end with no code change or deployment.		
Automated by	`ResourceTypesTest`: "Inventory.Type.NoCode: a new type is a row, listed at once with its attributes"		

TC-INV-04	**Resource Owner adds, edits, and retires an Exclusive Resource**		
Requirements	**Inventory.Resource.Add**		
Actor	`dave` (Resource Owner); `alice` where named		
Priority	High		
Preconditions	Test world.		
Test data	Resource R1: Meeting room “Room C”, seats 6, wing North.		
Step	**Action**	**Input**	**Expected result**
1	Choose Add Resource in the Inventory and fill in R1.	R1	Room C is listed under Meeting room with its attributes, owner Dave Kim, and 1 unit.
2	Add a second Meeting room with an attribute value outside its list.	Name “Room D”, wing East	Refused with `ATTRIBUTE_INVALID`; Room D is not added.
3	Add another Meeting room with the name of an existing one.	Name “Room A”	Refused with `RESOURCE_NAME_TAKEN`.
4	Edit Room C.	seats 10	Room C shows 10 seats.
5	Retire Room C.	None	Room C is marked retired and is no longer offered for new lines.
6	As `alice`, call the API to retire Room A.	POST `/api/inventory/resources/{id}/retire`	Refused with `FORBIDDEN`; Room A is unchanged.
Pass criteria	Only the Resource Owner changed the Inventory, and only valid values were saved.		
Automated by	`ResourcesTest`: “An Exclusive Resource has units 1 and the caller as its owner”, “A Resource’s name is unique within its type”, “A retired Resource is never offered again and cannot change”; `InventorySecurityTest`		

TC-INV-05	**Pool Capacity of a Metered type is the sum of its Resources’ units**		
Requirements	**Inventory.Resource.Add**		
Actor	`dave` (Resource Owner)		
Priority	High		
Preconditions	Test world: GPU Pool Capacity 16.		
Test data	Resource R2: GPU “gpu-node-c”, 4 units, model A100.		
Step	**Action**	**Input**	**Expected result**
1	Open GPU in the Inventory.	None	Pool Capacity 16 units.
2	Add R2.	R2	Pool Capacity 20 units.
3	Change the units of gpu-node-c.	6 units	Pool Capacity 22 units.
4	Retire gpu-node-c.	None	Pool Capacity 16 units.
Pass criteria	The Pool Capacity always equals the sum of the units of the unretired GPU Resources.		
Automated by	`ResourcesTest`: “Pool Capacity is the sum of the units of the unretired Resources”		

TC-INV-06 A reduction or retirement below booked capacity is refused, naming the earliest hour			
Requirements	**Inventory.Resource.Add**		
Actor	`dave` (Resource Owner)		
Priority	High		
Preconditions	`alice` has an Allocated Request A1 with a line for 12 GPU units, D 10:00–14:00, and an Allocated Request A2 with Room A, D 10:00–12:00.		
Test data	GPU Pool Capacity 16 (gpu-node-a 8, gpu-node-b 8).		
Step	**Action**	**Input**	**Expected result**
1	Reduce gpu-node-a.	2 units (capacity would be 10)	Refused with `UNITS_BELOW_USAGE`, naming D 10:00 as the earliest hour; gpu-node-a keeps 8 units.
2	Reduce gpu-node-a.	4 units (capacity would be 12)	Accepted; Pool Capacity 12.
3	Retire gpu-node-b.	None	Refused with `UNITS_BELOW_USAGE`, naming D 10:00.
4	Retire Room A.	None	Refused with `RESOURCE_IN_USE`, naming D 10:00, the start of A2's allocation.
5	Retire Room B, which has no future booking.	None	Accepted.
Pass criteria	No allocation of A1 or A2 lost capacity; only the reductions that leave the peak of future usage were accepted.		
Automated by	`ReductionGuardTest`; `UsageReductionGuardTest`: "refuses a reduction below the peak, naming the earliest failing hour, not the first used one"		

TC-INV-08 Resource timeline shows free, on Hold, and allocated parts with their holders			
Requirements	**Inventory.View.Holder**, **Lifecycle.Hold.Place**		
Actor	`ben` (Member); `dave` where named		
Priority	High		
Preconditions	`alice` has an Allocated Request A1 with Room A, D 10:00–12:00. `bob` has a Pending Request B1 with a Matched line (Java, quantity 1, days D to D+1) and Room A, D 14:00–16:00, so Room A is on Hold for B1.		
Test data	Window D 08:00–18:00.		
Step	**Action**	**Input**	**Expected result**
1	Open Room A and show its timeline for the window.	D 08:00–18:00	08:00–10:00 free; 10:00–12:00 allocated to Platform, Request A1; 12:00–14:00 free; 14:00–16:00 on Hold for Research, Request B1; 16:00–18:00 free.
2	As `dave`, check the notifications and the Request screens.	None	`dave` has no approval request for A1 or B1: a Resource Owner takes no part in allocation.
Pass criteria	Every part of the window shows the right status and, where held or allocated, the Team and Request.		
Automated by	None yet		

In the full edition only: **TC-INV-02** Type name, class, and the built-in Person type are protected; **TC-INV-03** A retired type accepts no new lines and is never deleted; **TC-INV-07** Filter Resources by type, attribute value, and being free for a window.

4.2 Member Profiles

These cases establish that a Member's skills come only from the Administrator's list, that availability and Loans together decide when a Member can be lent, and that profiles are visible only to the Member and their own Manager.

TC-PRF-01	**A Member maintains their own skills from the skill list**		
Requirements	**Profile.Skills**		
Actor	`ben` (Member)		
Priority	High		
Preconditions	Test world with the Matched data: Ben holds Java.		
Test data	Skill list: Statistics, Java, Microscopy.		
Step	**Action**	**Input**	**Expected result**
1	Open My profile.	None	Shows name Ben Ortiz, email, Team Platform, skill Java, and his availability range.
2	Add a skill.	Statistics (chosen from the list)	Skills Java and Statistics.
3	Try to add a skill that is not on the list.	"Kotlin" typed as free text	No free text is accepted: the field offers only the list's skills, and the API refuses an unknown skill with `VALIDATION_FAILED`.
4	Try to change the Team on the profile.	Team Research	The Team is shown but not editable; the API refuses the change with `FORBIDDEN`.
5	As `alice`, open a Draft line for Person with skill Statistics, quantity 1, days D to D+1, and submit.	None	Ben is not on the Shortlist: he is in the requesting Team (see TC-MAT-01).
Pass criteria	Ben's skills are exactly Java and Statistics, and his Team is Platform.		
Automated by	`ProfilesTest`: "prf-skills: a Member maintains their own skills from the skill list, and not their Team" (steps 1–4; the Shortlist step: none yet)		

TC-PRF-02	**Availability ranges decide whether a Member is available for a window**		
Requirements	**Profile.Availability**, **Matched.Shortlist.Build**		
Actor	`dave` (Member); `alice` where named		
Priority	High		
Preconditions	Test world with the Matched data.		
Test data	Dave's availability replaced by two ranges: day of execution to D, and D+2 to D+60. Line M1: Person, skill Statistics, quantity 1, days D to D+1. Line M2: same, days D+2 to D+3.		
Step	**Action**	**Input**	**Expected result**
1	On My profile, replace the availability range by the two ranges.	The two ranges	Both ranges are listed.
2	As `alice`, submit a Request with M1 and open its Shortlist.	M1	Rita Gomez only: D+1 is outside Dave's ranges, so he is not available for the whole window.
3	As `alice`, submit a second Request with M2 and open its Shortlist.	M2	Dave Kim, then Rita Gomez: every day of M2 lies inside a range.
Pass criteria	A Member is a Candidate only when every day of the window lies inside one of their ranges.		
Automated by	`ProfilesTest`: "prf-availability: a Member is available only when one range holds every day of the window" (step 1, and availability for M1 and M2 at the service level; the Shortlist steps: none yet)		

TC-PRF-03 A Member with an overlapping Loan is not a Candidate, and a Loan leaves availability unchanged			
Requirements	**Profile.Availability**, **Matched.Shortlist.Build**, **Release.Loan.Prorate**		
Actor	`alice` (Manager, Platform); `dave` where named		
Priority	High		
Preconditions	`bob` has an Allocated Request B1 in which Dave Kim is lent to Research for days D to D+1, formed as in TC-MAT-10.		
Test data	Line M1: Person, skill Statistics, quantity 1, days D+1 to D+2. Line M2: same, days D+2 to D+3.		
Step	**Action**	**Input**	**Expected result**
1	As `dave`, open My profile.	None	The availability range is unchanged (day of execution to D+60) and the Loan to Research for D to D+1 is listed.
2	Submit a Request with M1 and open its Shortlist.	M1	Rita Gomez only: Dave's Loan overlaps D+1.
3	Submit a Request with M2 and open its Shortlist.	M2	Dave Kim and Rita Gomez are both Candidates: the Loan does not touch D+2 or D+3. (Their order is checked in TC-MAT-02.)
4	As `bob`, end Dave's Loan early with last day D.	New last day D	The Loan now covers D only; Dave's availability is still unchanged.
5	Submit a new Request with M1 and open its Shortlist.	M1	Dave Kim is a Candidate again: he is free from D+1, the day after the Loan's new last day.
Pass criteria	A Person is never in two overlapping Loans, and Loans never change availability ranges.		
Automated by	None yet		

In the full edition only: **TC-PRF-04** Profiles are seen by the Member and their own Manager only.

4.3 Suggestions

These cases establish that a Member can propose one line to their Manager, that the proposal has no effect until the Manager submits it as part of a Request, and that accepting or declining it is recorded without ever losing the original ask.

TC-SUG-01 A Member creates a Suggestion that has no effect until submitted			
Requirements	**Suggest.Create**		
Actor	`ben` (Member, Platform); `alice` where named		
Priority	High		
Preconditions	Test world.		
Test data	Suggestion S1: Meeting room, quantity 1, D 10:00–12:00, note "Design review".		
Step	**Action**	**Input**	**Expected result**
1	Open Suggestions and create a Suggestion.	S1	S1 is listed as Open, addressed to Alice Chen, his Manager; the form allowed one line only and offered no choice of addressee.
2	Open Room A and Room B in the Inventory for D 10:00–12:00.	None	Both are free: no Hold was placed.
3	As `alice`, open the Statement and the Suggestions list.	None	Available Balance still 5000.00 and no new Transaction; S1 is listed first as Open, with Ben as its author.
4	As `ben`, try to create a Suggestion with two lines through the API.	Two lines	Refused with `VALIDATION_FAILED`.
Pass criteria	S1 exists as Open and the Inventory, Holds, and Ledger are unchanged.		
Automated by	None yet		

TC-SUG-02	The Manager accepts Suggestions as a new line and into an existing line		
Requirements	**Suggest.Accept.Existing**		
Actor	`alice` (Manager, Platform); `ben` where named		
Priority	High		
Preconditions	`ben` has created S1 (TC-SUG-01) and S2: Meeting room, quantity 1, D 10:00–12:00, note "Second room". S3: GPU, 2 units, D 10:00–12:00. All are Open.		
Test data	A new Draft R1 of Platform.		
Step	**Action**	**Input**	**Expected result**
1	Accept S1 into R1 as a new line.	S1, Draft R1	R1 has line L1 pre-filled from S1 (Meeting room, quantity 1, D 10:00–12:00) with its Quote 30.00 (Q1). S1 is Accepted and links to L1.
2	Accept S2 into the existing line L1.	S2, line L1	S2 is Accepted and links to L1; L1 lists both S1 and S2.
3	Try to accept S3 into L1.	S3, line L1	Refused with `TYPE_MISMATCH`: GPU is not Meeting room; S3 stays Open.
4	Edit L1.	Quantity 2	L1 now asks for 2 rooms; S1 and S2 still show their original ask of quantity 1 and still link to L1.
5	Try to accept S1 again.	S1	Refused with `SUGGESTION_CLOSED`: only Open Suggestions change state.
6	As `ben`, open Suggestions.	None	S1 and S2 Accepted, S3 Open.
Pass criteria	Every Accepted Suggestion records its line, and the original asks are unchanged.		
Automated by	None yet		

In the full edition only: **TC-SUG-03** The Manager declines a Suggestion with a reason; it is kept.

4.4 Draft and Submit a Request

These cases establish that only a Manager drafts, submits, and withdraws their own Team's Requests; that every line is checked as it is added and priced while drafting; that a Request the Team cannot afford is refused at submission with its shortfall; and that submission locks every Quote.

TC-DRF-01	A Manager creates a Draft for their own Team		
Requirements	**Draft.Create**, **Lifecycle.States**		
Actor	`alice` (Manager, Platform); `ben` where named		
Priority	High		
Preconditions	Test world.		
Test data	None.		
Step	**Action**	**Input**	**Expected result**
1	Open Requests and create a Request.	None	The Draft editor opens for a new Request of Team Platform in state Draft; its history shows "Created" by Alice Chen with a timestamp.
2	Return to Requests.	None	The new Draft is listed with Team Platform and state Draft.
3	Sign in as `ben` and look for Requests.	None	The menu has no Requests entry, and opening `/requests` shows the not-allowed page.
4	As `ben`, create a Request through the API.	POST `/api/requests`	Refused with `FORBIDDEN`; no Request is created.
Pass criteria	Exactly one Request exists, a Draft of Platform created by Alice.		
Automated by	`DraftRequestTest`: "Draft.Create: a Manager's new Draft belongs to their Team and is recorded in its history"; `MembersControllerTest`: "SE-roles: a caller without the role a service needs gets 403 FORBIDDEN"		

TC-DRF-02	Another Team's Request is not found		
Requirements	**Draft.Create**		
Actor	`bob` (Manager, Research)		
Priority	High		
Preconditions	`oscar` has an Allocated Request O1 with Room A, D 10:00–12:00. `alice` has a Draft A1 with one line, Meeting room, D 14:00–16:00, and a Waiting Request A2 whose line for Room A, D 10:00–12:00 waits behind O1.		
Test data	The ids of A1 and A2.		
Step	**Action**	**Input**	**Expected result**
1	Open Requests.	None	Neither A1 nor A2 is listed.
2	Open A1's detail, add a line to A1, and submit A1 through the API.	A1's id	Each answers `NOT_FOUND`.
3	Withdraw A2 through the API.	A2's id	`NOT_FOUND`; A2 stays Waiting.
Pass criteria	A1 and A2 are unchanged and nothing reveals that they exist.		
Automated by	`RequestLifecycleTest`: "Another Team's Request is not found: submit, withdraw, and the detail all answer 404"		

TC-DRF-03	Exclusive and Metered lines show their Quotes and the Draft its sums		
Requirements	**Draft.Line.Add**, **Draft.Quote.Line**, **Ledger.Price.Formula**		
Actor	`alice` (Manager, Platform)		
Priority	High		
Preconditions	`alice` has an empty Draft.		
Test data	L1: Meeting room, quantity 1, D 10:00–12:00 (Q1 = 30.00). L2: GPU, 4 units, D 16:00–18:00 (Q8 = 50.00). L3: Person, skill Statistics, quantity 1, days D to D+1.		
Step	**Action**	**Input**	**Expected result**
1	Add L1.	L1	L1 listed as Required with Allocation Class Exclusive (from the type) and Quote 30.00.
2	Add L2.	L2	L2 listed as Required, Metered, Quote 50.00. The form offered no Specific Resource and no Priority mark for GPU.
3	Add L3.	L3	L3 listed as Required, Matched, with no Quote.
4	Read the totals panel.	None	Required lines 80.00; Optional lines 0.00; Available Balance 5000.00; no shortfall.
Pass criteria	Each Quote equals its reference Quote, and Matched lines cost nothing.		
Automated by	`DraftRequestTest`: "Draft.Quote.Line and Draft.Quote.Totals: . . . "; `RequestPage.test.tsx`		

TC-DRF-05	Invalid lines are refused as they are added		
Requirements	**Draft.Line.Add**		
Actor	`alice` (Manager, Platform)		
Priority	High		
Preconditions	`alice` has an empty Draft. The Resource Type 3D printer exists and is retired (TC-INV-03).		
Test data	As in each step.		
Step	**Action**	**Input**	**Expected result**
1	Add a line whose window starts in the past.	Meeting room, 1, yesterday 10:00–12:00	Refused with `WINDOW_NOT_FUTURE`.
2	Add a line whose window ends before it starts.	Meeting room, 1, D 12:00–10:00	Refused with `WINDOW_INVALID`.
3	Add a line with quantity zero.	Meeting room, 0, D 10:00–12:00	Refused with `QUANTITY_INVALID`.
4	Add a line of a retired type.	3D printer, 1, D 10:00–12:00	Refused with `TYPE_RETIRED`.
5	Add a Metered line naming a Resource.	GPU, gpu-node-a, 4 units	Refused with `NAMED_RESOURCE_NOT_OFFERED`.
6	Add an Exclusive line naming a Resource of another type.	Meeting room, Microscope 1	Refused with `TYPE_MISMATCH`.
Pass criteria	The Draft is still empty, and each refusal shows its message beside the field it concerns.		
Automated by	`DraftRequestTest`: the "Draft.Line.Validate" tests, "A window that ends before it starts ... WINDOW_INVALID", "A named Resource of another type ... TYPE_MISMATCH", "A Metered line cannot name a Resource ... "		

TC-DRF-06	Lines are Required by default and may be split to make a part Optional		
Requirements	**Draft.Line.Required**, **Draft.Quote.Line**		
Actor	`alice` (Manager, Platform)		
Priority	High		
Preconditions	`alice` has an empty Draft.		
Test data	L1: Meeting room, quantity 2, D 10:00–12:00 (Quote 60.00). L2: GPU, 6 units, D 10:00–12:00 (Q11 = 90.00).		
Step	**Action**	**Input**	**Expected result**
1	Add L1 and L2.	L1, L2	Both Required; Required lines 150.00; Optional lines 0.00.
2	Split L1, moving one room to a new line.	1 of 2	L1 has quantity 1 and Quote 30.00, Required; a new line L1b of the same type and window has quantity 1, Quote 30.00, Optional. Required 120.00; Optional 30.00.
3	Split L2, moving 2 units.	2 of 6	L2 keeps 4 units (60.00), Required; L2b has 2 units (30.00), Optional. Required 90.00; Optional 60.00.
4	Try to split L1 again, moving its only room.	1 of 1	Refused with `LINE_NOT_ATOMIC`: both parts must keep at least one.
5	Mark L1b Required.	Required	Required 120.00; Optional 30.00.
Pass criteria	No line offers a partial-acceptance setting; only whole lines are Required or Optional.		
Automated by	`DraftCasesTest`: "drf-required-optional: Required by default; a split moves a part to an Optional line; marking it Required again"; `DraftRequestTest`: "Draft.Line.Split: ... " (both), "Split refuses with 422 LINE_NOT_ATOMIC unless both parts keep at least 1"		

TC-DRF-07	Submission refused while the Required sum exceeds the Available Balance		
Requirements	**Draft.Quote.Line**, **Ledger.Account**		
Actor	`bob` (Manager, Research)		
Priority	High		
Preconditions	Test world: Research's Available Balance is 600.00. `bob` has an empty Draft R1.		
Test data	L1: Confocal microscope, quantity 1, D 09:00–17:00 (Q9 = 480.00), Required. L2: Confocal microscope, quantity 1, D+1 09:00–12:00 (Q10 = 180.00), Required.		
Step	**Action**	**Input**	**Expected result**
1	Add L1 and L2.	L1, L2	Required lines 660.00; Available Balance 600.00; shortfall 60.00 shown.
2	Submit R1.	None	Refused with `INSUFFICIENT_BALANCE` and the message "short by 60.00 credits"; R1 stays a Draft.
3	Mark L2 Optional and submit R1.	L2 Optional	Required 480.00 is within 600.00, so submission is accepted; the Optional line does not count toward the refusal.
4	Open the Statement of Research.	None	One Charge of 480.00 for L1. L2 was not affordable after L1 was charged, so it is Dropped and charged nothing (TC-LIF-09).
Pass criteria	After step 2 the Statement and Inventory were unchanged; after step 4 Research's Balance is 120.00.		
Automated by	`RequestLifecycleTest`: "INSUFFICIENT_BALANCE: a Request costing more than the Available Balance is refused with the shortfall . . . "; `DraftRequestTest`: "Draft.Quote.Totals: the shortfall is the Required sum above the Available Balance; Optional lines do not count"		

TC-DRF-09	Submission locks every Quote and records the submission time		
Requirements	**Draft.Submit.Lock**, **Planner.Settings**		
Actor	`alice` (Manager, Platform); `erin` where named		
Priority	High		
Preconditions	`alice` has a Draft R1 with L1: Room A, D 10:00–12:00 (30.00), and a Draft R2 with L2: Room B, D 14:00–16:00 (30.00).		
Test data	New Unit Price for Meeting room: 20.00.		
Step	**Action**	**Input**	**Expected result**
1	Submit R1.	None	R1 is Allocated; its history shows Submitted with the submission time, then Allocated. L1's Quote 30.00 is shown as locked.
2	As `erin`, set the Unit Price of Meeting room.	20.00	Saved, and recorded with actor, old value 10.00, new value 20.00, and timestamp.
3	Open R1 and the Statement.	None	L1's locked Quote is still 30.00; the Charge for L1 is 30.00.
4	Open R2.	None	L2's Quote is now 60.00, since R2 is still a Draft.
Pass criteria	A locked Quote never changed, and the Charge equals it.		
Automated by	`RequestLifecycleTest`: "Draft.Submit.Lock: the Quotes are locked at submission and do not follow a later price change"		

TC-DRF-10	Withdrawing from Draft, Waiting, or Pending releases everything held		
Requirements	**Draft.Create**, **Lifecycle.Hold.Place**		
Actor	`bob` (Manager, Research)		
Priority	High		
Preconditions	`alice` has an Allocated Request with Room A, D 10:00–12:00. `bob` has: a Draft B1 (any line); a Waiting Request B2 with Room A, quantity 1, Specific Resource Room A, D 10:00–12:00 at waitlist position 1; a Pending Request B3 with a Matched line (Java, quantity 1, days D to D+1) and Microscope 1, D 09:00–17:00 on Hold (held credits 480.00).		
Test data	None.		
Step	**Action**	**Input**	**Expected result**
1	Withdraw B1.	None	B1 is Withdrawn; editing or submitting it is refused with `WRONG_STATE`.
2	Withdraw B2.	None	B2 is Withdrawn; its line leaves the waitlist for Room A.
3	Open the Statement before and after withdrawing B3.	None	Before: Available Balance 120.00 (480.00 held). After: B3 Withdrawn, Available Balance 600.00, no Transaction posted.
4	Open Microscope 1's timeline for D.	D 09:00–17:00	Free: the Hold has been released.
5	Try to withdraw `alice`'s Allocated Request through the API as `alice`.	Its id	Refused with `WRONG_STATE`: an Allocated Request is released line by line, not withdrawn.
Pass criteria	No Hold, held credit, or waitlist entry of B1 to B3 remains.		
Automated by	`RequestLifecycleTest`: "Draft.Create: withdrawing a Waiting Request cancels its lines . . . ", "Draft.Create: a Draft can be withdrawn . . . ", "WRONG_STATE: . . . an Allocated one cannot be withdrawn" (Pending part: none yet)		

In the full edition only: **TC-DRF-04** Line options by Allocation Class; **TC-DRF-08** A Required sum equal to the Available Balance is accepted; **TC-DRF-11** Editing a Waiting Request returns it to Draft and requotes it.

4.5 Matched Allocation

These cases establish that the Shortlist contains exactly the Members of other Teams who have the skills, the availability, and no overlapping Loan, in the stated order; that the requesting Manager selects Candidates and each lending Manager decides; that refusals and silence are handled without restarting the Objection Window; and that an approved Candidate becomes a Loan with its Earning.

TC-MAT-01	**The Shortlist holds the qualified Members of other Teams**		
Requirements	**Matched.Shortlist.Build**, **Lifecycle.Evaluate**		
Actor	`alice` (Manager, Platform)		
Priority	High		
Preconditions	Test world with the Matched data.		
Test data	M1: Person, skill Statistics, quantity 1, days D to D+1, Required.		
Step	**Action**	**Input**	**Expected result**
1	Create a Draft with M1 and submit it.	M1	The Request is Pending: M1 needs an approval.
2	Open M1's Shortlist.	None	Dave Kim (Operations, Statistics, 0 person-days lent), then Rita Gomez (Research, Statistics and Microscopy, 0). Ben Ortiz is absent: he is in Platform and lacks Statistics. No Candidate is selected yet.
3	Check what each row shows.	None	Name, Team, skills, and person-days lent only; no link to the Candidate's other Requests or Statements.
Pass criteria	The Shortlist has exactly two Candidates in the stated order, and nothing was selected automatically.		
Automated by	None yet		

TC-MAT-03	**The Manager selects exactly the line's quantity; a short Shortlist leaves the Request Waiting**		
Requirements	**Matched.Shortlist.Build**, **Matched.Approve.Route**		
Actor	`bob` (Manager, Research); `alice` where named		
Priority	High		
Preconditions	Test world with the Matched data.		
Test data	M2: Person, skill Statistics, quantity 2, days D to D+1, Required. M1: Person, skill Statistics, quantity 1, days D to D+1 (for `alice`).		
Step	**Action**	**Input**	**Expected result**
1	Submit a Request with M2 and open its Shortlist.	M2	Dave Kim only, since Rita is in Research. The Request is Waiting: M2 is Required and has fewer Candidates than its quantity.
2	As `alice`, submit a Request with M1 and try to select both Candidates.	Dave Kim and Rita Gomez	Refused with `LINE_NOT_ATOMIC`: exactly one Candidate must be selected.
3	As `alice`, try to select a Member who is not on the Shortlist through the API.	Carol Davis (Operations, no skills)	Refused with `NOT_A_CANDIDATE`.
4	As `alice`, select one Candidate.	Dave Kim	Dave is selected and an approval request is sent; Dave receives a notification that he was selected.
Pass criteria	No Candidate was selected without the Manager's action.		
Automated by	None yet		

TC-MAT-04	The approval request goes to the Candidate's Manager, who decides		
Requirements	**Matched.Approve.Route**		
Actor	`oscar` (Manager, Operations); `alice` where named		
Priority	High		
Preconditions	`alice` has a Pending Request A1 with M1 (Statistics, quantity 1, days D to D+1) and Dave Kim selected.		
Test data	Reason "Dave is free that week".		
Step	**Action**	**Input**	**Expected result**
1	Open Approvals.	None	One approval request: Candidate Dave Kim, requesting Team Platform, days D to D+1, Earning 800.00 (EA).
2	As `alice`, try to decide the approval through the API.	Approve	Refused with `FORBIDDEN`: the requesting Manager cannot decide.
3	Approve with a reason.	Reason	The decision is recorded with actor Oscar Reed, the reason, and the timestamp. A1 becomes Allocated (its only line is approved).
4	Try to decide it again.	Refuse	Refused with `ALREADY_DECIDED`.
Pass criteria	Exactly one approval request was sent, to Oscar, and only his decision counted.		
Automated by	None yet		

TC-MAT-06	A refusal asks for a replacement without restarting the Objection Window		
Requirements	**Matched.Refuse.Replace**, **Lifecycle.Waiting.Reevaluate**, **Lifecycle.Window.Start**		
Actor	`alice` (Manager, Platform); `oscar` and `bob` where named		
Priority	High		
Preconditions	`alice` has a Pending Request A1 with M1 (Statistics, quantity 1, days D to D+1, Required, Dave Kim selected) and L1: Room A, D 10:00–12:00, on Hold (30.00 held). A1's Objection Window deadline is shown on A1, say W.		
Test data	Reason "Dave is on another project".		
Step	**Action**	**Input**	**Expected result**
1	As `oscar`, refuse the approval with a reason.	Reason	Recorded with actor, reason, and timestamp.
2	Open A1.	None	A1 is Waiting; L1's Hold is released and the 30.00 is no longer held; A1 asks Alice to select a replacement for M1; the deadline is still W.
3	Open M1's Shortlist and select a replacement.	Rita Gomez	A1 returns to Pending; L1 is on Hold again; an approval request goes to Bob Patel; the deadline is still W.
4	As `bob`, approve.	None	A1 is Allocated; Rita has a Loan to Platform for D to D+1.
Pass criteria	The Objection Window never restarted, and Dave Kim has no Loan.		
Automated by	None yet		

TC-MAT-07	A refusal with no Candidate left refuses a Required line and drops an Optional one		
Requirements	**Matched.Refuse.Replace**, **Lifecycle.States**, **Lifecycle.Evaluate**		
Actor	`bob` (Manager, Research); `oscar` and `alice` where named		
Priority	High		
Preconditions	Test world with the Matched data.		
Test data	Request B1: M4 Person, named Person Dave Kim, quantity 1, days D to D+1, Required; L1 Room A, D 10:00–12:00, Required. Request B2: L2 Room B, D 10:00–12:00, Required; M5 Person, skill Statistics, quantity 1, days D to D+1, Optional.		
Step	**Action**	**Input**	**Expected result**
1	Submit B1. As `oscar`, refuse the approval for Dave.	None	B1 is Refused: the line named a Person, so no replacement is possible. L1's Hold is released; nothing is charged; Bob is notified.
2	Submit B2.	None	B2 is Allocated at once: no Required line needs approval. L2 is Allocated and charged; M5 stays open, awaiting a Candidate.
3	Select Dave Kim for M5. As `oscar`, refuse.	Dave Kim	The Shortlist has no other Candidate, so M5 is Dropped with its reason; B2 stays Allocated with L2 only.
4	Open Research's Statement.	None	One Charge of 30.00 for L2; nothing for B1.
Pass criteria	B1 is final in Refused; B2 was Allocated at submission, and M5 was Dropped only when it was refused with no Candidate left.		
Automated by	None yet		

TC-MAT-08	A silent Manager's approval escalates to the Capacity Planner		
Requirements	**Matched.Escalate**, **Lifecycle.Window.Start**, **Planner.Settings**		
Actor	`erin` (Capacity Planner); `alice` and `oscar` where named		
Priority	High		
Preconditions	`alice` has a Pending Request A1 with M1 (Statistics, quantity 1, days D to D+1, Dave Kim selected) and L1: Room A, D 10:00–12:00, on Hold. Oscar has not decided.		
Test data	Objection Window two working days from A1's submission.		
Step	**Action**	**Input**	**Expected result**
1	[clock] Let the Objection Window lapse.	Two working days	Within five minutes: A1 is Waiting, L1's Hold is released, and A1 is escalated; Alice and Erin are notified.
2	Open Escalations.	None	A1 is listed with its outstanding approval (Dave Kim), the silent Manager Oscar Reed, and the time since the lapse.
3	Approve on Oscar's behalf.	Approve	Recorded with actor Erin Walsh. A1 is re-evaluated: Room A is still free, so A1 is Allocated at its locked Quotes. Oscar is notified of the outcome.
4	As `oscar`, open Approvals.	None	The request is no longer open there; it shows as decided by the Capacity Planner.
Pass criteria	Exactly one decision was recorded, by Erin, and A1 is Allocated.		
Automated by	None yet		

TC-MAT-10	An approved Candidate becomes a Loan and the lending Team earns		
Requirements	**Matched.Loan.Form**, **Ledger.Earning.Post**		
Actor	`bob` (Manager, Research); `oscar` and `dave` where named		
Priority	High		
Preconditions	Test world with the Matched data.		
Test data	M1: Person, skill Statistics, quantity 1, days D to D+1, Required.		
Step	**Action**	**Input**	**Expected result**
1	Submit a Request B1 with M1 and select Dave Kim.	Dave Kim	B1 is Pending. Dave is notified that he was selected.
2	As `oscar`, approve.	None	B1 is Allocated; a Loan of Dave Kim to Research for D to D+1 is formed.
3	Open the Statements of Operations (as `oscar`) and Research (as `bob`).	None	Operations: an Earning of 800.00 (2 person-days × 400.00) for B1's line, earned credits 800.00. Research: no Charge for M1.
4	As `dave`, open the notifications and My profile.	None	A notification that the Loan formed; the Loan to Research for D to D+1 is listed; there is no action to accept or decline it.
Pass criteria	One Loan and one Earning of 800.00 exist for B1.		
Automated by	None yet		

In the full edition only: **TC-MAT-02** The Shortlist is ordered by person-days lent in the current Period, then by name; **TC-MAT-05** No approval is routed to the requesting Manager for their own Team; **TC-MAT-09** After the lapse, a later approval goes straight to the Capacity Planner; **TC-MAT-11** An Optional Matched line approved after allocation forms its Loan then.

4.6 Exclusive Allocation

These cases establish that an Exclusive instance is never given to two overlapping lines; how the instance is chosen; that unsatisfiable Required lines wait in the stated order and are filled when capacity frees; and that a Priority line Bumps standard lines only within the stated limits, with a full Refund for the Bumped line.

TC-EXC-01	An instance is never given to two overlapping windows		
Requirements	**Exclusive.Conflict**, **Lifecycle.Evaluate**		
Actor	`bob` (Manager, Research)		
Priority	High		
Preconditions	`alice` has an Allocated Request A1 with Room A, D 10:00–12:00.		
Test data	L1: Room A (named), D 11:00–13:00. L2: Room A (named), D 12:00–14:00.		
Step	**Action**	**Input**	**Expected result**
1	Submit a Request B1 with L1.	L1	B1 is Waiting; L1 is at waitlist position 1 with the reason that no instance is free.
2	Submit a Request B2 with L2.	L2	B2 is Allocated: a window starting when another ends does not overlap.
3	Open Room A's timeline for D.	D 08:00–18:00	10:00–12:00 Platform (A1); 12:00–14:00 Research (B2); nothing for B1.
Pass criteria	No two allocations or Holds of Room A overlap.		
Automated by	`ExclusiveAllocatorTest`: "An overlapping window finds no free instance; back-to-back windows both fit"; `ExclusionConstraintTest`; `RequestLifecycleTest`: "Exclusive.Waitlist.Join: Room A at an overlapping time waits at position 1 . . . "		

TC-EXC-04	Waitlist order: Bumped first, then Priority, then submission time		
Requirements	**Exclusive.Waitlist.Join**		
Actor	`bob`, `alice`, `oscar` (Managers) as named		
Priority	High		
Preconditions	Room A and Room B are both allocated for D 10:00–12:00 to Priority lines of `oscar`, so no line in this case can Bump them (TC-EXC-10).		
Test data	Each line: Meeting room, quantity 1, D 10:00–12:00, Required unless stated.		
Step	**Action**	**Input**	**Expected result**
1	As `bob`, submit B1 (standard).	None	B1 Waiting; its line at position 1.
2	As `alice`, submit A1 (standard).	None	A1 Waiting at position 2.
3	As `alice`, submit A2 with the line marked Priority.	Priority	A2 Waiting at position 1, ahead of all standard lines; B1 moves to 2 and A1 to 3.
4	As `bob`, submit B2 with the room line Optional and a Required line GPU, 4 units, D 10:00–12:00.	Optional room line	B2 is Allocated with its GPU line; the room line is Dropped with its reason, not waitlisted.
5	Open B1 as `bob`.	None	B1's line shows position 2.
Pass criteria	Positions are A2, B1, A1; no Optional line is on the waitlist.		
Automated by	`WaitlistTest`: "Positions follow submission time among overlapping windows . . . ", "Bumped lines come first, then Priority lines, then by submission time", ". . . Optional lines never join"		

TC-EXC-05	Freed capacity goes to the first waiting line whose window fits		
Requirements	**Exclusive.Waitlist.Join**, **Lifecycle.Waiting.Reevaluate**, **Release.Line**		
Actor	`alice` (Manager, Platform); `bob` and `oscar` where named		
Priority	High		
Preconditions	Allocated: `alice`'s A1 with Room A, D 10:00–12:00, and `oscar`'s O1 with Room A, D 12:00–14:00. Waiting, in this order: `bob`'s B1 with Room A (named), D 10:00–13:00; `oscar`'s O2 with Room A (named), D 10:00–11:00.		
Test data	None.		
Step	**Action**	**Input**	**Expected result**
1	Release A1's line in whole.	None	The line is Released and a Refund is posted (TC-REL-01).
2	Open B1 as `bob` and O2 as `oscar`.	None	B1 is still Waiting at position 1: its window needs 12:00–13:00, which O1 still holds. O2, the next line, fits and is Allocated on Room A for D 10:00–11:00; Oscar is notified that it was satisfied from the waitlist.
Pass criteria	The freed slot was offered in waitlist order and given to the first line whose window fits.		
Automated by	None yet		

TC-EXC-07	A Priority line Bumps the latest-submitted blocking standard line		
Requirements	**Exclusive.Priority.Bump**, **Ledger.Price.Formula**		
Actor	`alice` (Manager, Platform)		
Priority	High		
Preconditions	Standard lines, all Allocated: `bob`'s B1 with Room A, D 10:00–12:00 (submitted first); `oscar`'s O1 with Room B, D 10:00–12:00 (submitted second).		
Test data	L1: Meeting room, quantity 1, D 10:00–12:00, Priority (Q4 = 60.00).		
Step	**Action**	**Input**	**Expected result**
1	Add L1 to a Draft.	L1	Quote 60.00, twice the standard Quote.
2	Submit.	None	The Request is Allocated with L1 on Room B. O1's line is Bumped: each room needed one Bump, and O1 was submitted later.
3	Open Room A's and Room B's timelines for D.	None	Room A: B1 unchanged. Room B: Platform, Priority line L1.
Pass criteria	Exactly one line was Bumped, the latest-submitted one; the Charge for L1 is 60.00.		
Automated by	None yet		

TC-EXC-08	A Bumped line is refunded in full, heads its waitlist, and returns its Request to Waiting		
Requirements	**Exclusive.Bump.Effect**, **Lifecycle.States**		
Actor	`oscar` (Manager, Operations)		
Priority	High		
Preconditions	O1 as in TC-EXC-07, with a second line O1b: GPU, 4 units, D 10:00–12:00 (60.00), both Required; O1 was Allocated with Charges 30.00 and 60.00 before Alice's Priority line Bumped its room line.		
Test data	None.		
Step	**Action**	**Input**	**Expected result**
1	Open the notifications.	None	A notification that O1's room line was Bumped, with the reason (a Priority line).
2	Open O1.	None	O1 is Waiting; the room line is Bumped and at position 1 of the waitlist for Meeting room, D 10:00–12:00; O1b is still allocated. The history records the Bump with both lines, the actor, and the timestamp.
3	Open the Statement of Operations.	None	A Refund of 30.00 referencing the room line's Charge; the GPU Charge of 60.00 is untouched; Balance 940.00 (1000.00 − 30.00 − 60.00 + 30.00).
Pass criteria	The Refund, the waitlist head, the state change, and the record were all made together.		
Automated by	None yet		

TC-EXC-09	No Bump inside the bump cutoff; the Priority line waits ahead of standard lines		
Requirements	**Exclusive.Priority.Bump**		
Actor	`alice` (Manager, Platform)		
Priority	High		
Preconditions	Standard lines Allocated: `bob`'s with Room A for H to H+2 hours and `oscar`'s with Room B for H to H+2 hours. `bob` also has a Waiting standard line for Meeting room, H to H+2 hours.		
Test data	L1: Meeting room, quantity 1, H to H+2 hours, Priority.		
Step	**Action**	**Input**	**Expected result**
1	Submit a Request with L1.	L1	No Bump: both blocking lines start within 24 hours. The Request is Waiting and L1 is at position 1, ahead of Bob's waiting standard line.
Pass criteria	Both blocking lines are still allocated.		
Automated by	None yet		

In the full edition only: **TC-EXC-02** Without a named Resource, the free instance with the fewest allocated hours is chosen; **TC-EXC-03** A recurring or multi-instance line needs the same free instances for every occurrence; **TC-EXC-06** A waiting line leaves the waitlist when its window starts; **TC-EXC-10** A Priority line is never Bumped and never Bumps another Priority line.

4.7 Metered Allocation

These cases establish that a Metered line is satisfied with all its units for every hour of its window or not at all; that the Pool Capacity and a Team's Quota are checked hour by hour; and, for the Stretch requirements, that service tiers, tier Bumps, and recorded usage behave as specified.

TC-MET-01	A Metered line asks for units of a type and is all or nothing		
Requirements	**Metered.Line.Form**, **Ledger.Price.Formula**		
Actor	`oscar` (Manager, Operations)		
Priority	High		
Preconditions	Test world: GPU Pool Capacity 16, no GPU allocated.		
Test data	L1: GPU, 4 units, D 16:00–18:00 (Q8 = 50.00). L2: GPU, 20 units, D 10:00–12:00.		
Step	**Action**	**Input**	**Expected result**
1	Add L1 to a Draft.	L1	Quote 50.00: $4 \times 5.00 \times 1.5$ for 16:00 plus $4 \times 5.00 \times 1.0$ for 17:00. The form offers no Specific Resource and no Priority mark.
2	Submit a Request with L1.	None	Allocated; the GPU timeline shows 4 units for Operations, D 16:00–18:00.
3	Submit a Request with L2.	L2	Waiting with `POOL_FULL`: 20 units exceed the Pool Capacity of 16; no part of the 20 units is allocated or held.
Pass criteria	Only L1's 4 units are allocated.		
Automated by	`MeteredAllocatorTest`: "Metered.Line.Form, Metered.Pool.Check: MeteredAllocator"; `PricingTest`		

TC-MET-02	Pool Capacity is checked for every hour of the window		
Requirements	Metered.Pool.Check		
Actor	`oscar` (Manager, Operations)		
Priority	High		
Preconditions	`alice` has an Allocated Request A1 with GPU, 12 units, D 10:00–14:00.		
Test data	O1: GPU, 6 units, D 13:00–15:00. O2: GPU, 4 units, D 13:00–15:00. O3: GPU, 6 units, D 14:00–16:00.		
Step	**Action**	**Input**	**Expected result**
1	Submit a Request with O1.	O1	Waiting with `POOL_FULL` at D 13:00: $12 + 6 = 18$ units exceed 16 in that hour.
2	Submit a Request with O2.	O2	Allocated: at 13:00, $12 + 4 = 16$ units, exactly the Pool Capacity.
3	Submit a Request with O3.	O3	Allocated: A1 ends at 14:00, so 14:00 and 15:00 carry $4 + 6 = 10$ units.
Pass criteria	No hour of D carries more than 16 allocated or held GPU units.		
Automated by	`MeteredAllocatorTest`: “is POOL_FULL at 14:00, one unit short”, “fits from 15:00, after A ends”; `AllocationConcurrencyTest`: “the last units of a pool go to exactly one line . . . ”		

TC-MET-03	A Team’s Quota is checked for every hour; other Teams’ units do not count		
Requirements	Metered.Pool.Check		
Actor	`bob` (Manager, Research)		
Priority	High		
Preconditions	Research has a Quota of 8 GPU units. `alice` has an Allocated Request with GPU, 2 units, D 10:00–12:00.		
Test data	B1: GPU, 6 units, D 10:00–12:00 (Q11 = 90.00). B2: GPU, 4 units, D 11:00–13:00. B3: GPU, 2 units, D 11:00–13:00 (30.00).		
Step	**Action**	**Input**	**Expected result**
1	Open a Draft and add B2’s line.	B2	The Draft shows Research’s Quota of 8 and the peak of 0 units already allocated or held in the window.
2	Submit a Request with B1.	B1	Allocated.
3	Submit a Request with B2.	B2	Waiting with `QUOTA_EXCEEDED` at D 11:00: Research would hold $6 + 4 = 10$ units, over its Quota, although the pool has room ($2 + 6 + 4 = 12$).
4	Submit a Request with B3.	B3	Allocated: Research holds $6 + 2 = 8$ units at 11:00; Platform’s 2 units do not count toward it.
Pass criteria	Research never holds more than 8 units in any hour.		
Automated by	`RequestLifecycleTest`: “Metered.Pool.Check: 10 GPU units exceed Research’s Quota of 8 . . . ”; `MeteredAllocatorTest`: “Only the requesting Team’s own units count toward its Quota”		

TC-MET-05	[Stretch] The Capacity Planner defines service tiers for a Metered type		
Requirements	**Metered.Tier.Define**		
Actor	`erin` (Capacity Planner); `alice` where named		
Priority	Low		
Preconditions	Test world.		
Test data	Tiers for GPU: Standard ×1.0, Express ×1.5. L1: GPU, 4 units, D 10:00–12:00, tier Express.		
Step	**Action**	**Input**	**Expected result**
1	In Settings, define the two tiers for GPU.	The tiers	Saved and recorded with actor, old and new value, and timestamp.
2	As `alice`, add L1 to a Draft.	L1	The line offers a tier choice; with Express the Quote is $60.00 \times 1.5 = 90.00$.
Pass criteria	The tier's premium multiplies the Unit Price.		
Automated by	None yet		

TC-MET-06	[Stretch] A higher-tier Metered line Bumps a lower-tier one		
Requirements	**Metered.Tier.Bump**		
Actor	`alice` (Manager, Platform); `oscar` where named		
Priority	Low		
Preconditions	Tiers of TC-MET-05 and a bump cutoff of 24 hours for GPU. `oscar` has an Allocated Standard line O1: GPU, 16 units, D 10:00–12:00 (240.00).		
Test data	L1: GPU, 4 units, D 10:00–12:00, tier Express.		
Step	**Action**	**Input**	**Expected result**
1	Submit a Request with L1.	L1	Allocated: O1 is Bumped to make room.
2	As `oscar`, open O1 and the Statement.	None	O1's line is Bumped, at the head of the GPU waitlist, with a notification; a Refund of 240.00, the full Charge.
Pass criteria	No hour exceeds the Pool Capacity, and the Bumped line is refunded in full.		
Automated by	None yet		

TC-MET-07	[Stretch] The Resource Owner records the units consumed, with no Ledger effect		
Requirements	**Metered.Usage.Record**, **Metered.Usage.Compare**		
Actor	`dave` (Resource Owner)		
Priority	Low		
Preconditions	`alice` had an Allocated GPU line of 4 units for E 10:00–12:00 (60.00), and the Period containing E has ended.		
Test data	Consumed: 3 units.		
Step	**Action**	**Input**	**Expected result**
1	Open the usage list for the ended Period.	The Period	Alice's allocation is listed with 4 units requested.
2	Record the units consumed.	3 units	Saved for that allocation.
3	As `alice`, open the Statement for that Period.	None	The Charge is still 60.00 and no Transaction was added.
Pass criteria	Usage is recorded and the Ledger is unchanged.		
Automated by	None yet		

In the full edition only: **TC-MET-04** A waiting Metered line is satisfied when its units fit in every hour.

4.8 Request Lifecycle and Holds

These cases establish that a submitted Request moves to Allocated, Pending, or Waiting by the stated rule, all or nothing; that Holds block capacity and reserve credits only while the Objection Window runs; that allocation commits everything in one step; and that a Waiting Request is allocated only when all its Required lines can be satisfied at once.

TC-LIF-01 A Request without Matched lines goes from Submitted straight to Allocated			
Requirements	**Lifecycle.Evaluate**, **Lifecycle.States**, **Lifecycle.Allocate**		
Actor	`alice` (Manager, Platform)		
Priority	High		
Preconditions	Test world.		
Test data	L1: Room A, D 10:00–12:00 (Q1 = 30.00). L2: GPU, 4 units, D 10:00–12:00 (Q7 = 60.00).		
Step	**Action**	**Input**	**Expected result**
1	Submit a Request A1 with L1 and L2.	L1, L2	A1 is Allocated; it never enters Pending.
2	Open A1's history.	None	Created, Submitted, and Allocated in that order, each with actor Alice Chen and a timestamp; both lines Allocated.
3	Open the Statement of Platform.	None	Charges of 30.00 and 60.00, each equal to its locked Quote; Balance 4910.00.
4	Open the notifications.	None	A notification that A1 was Allocated.
Pass criteria	A1 and both lines are Allocated and charged exactly once.		
Automated by	`RequestLifecycleTest`: "Lifecycle.Allocate: every Required line fits, so the lines are allocated and charged their locked Quotes"; `SubmitEndToEndTest`		

TC-LIF-02 If one Required line cannot be satisfied, nothing is held, allocated, or charged			
Requirements	**Lifecycle.Evaluate**, **Lifecycle.States**		
Actor	`bob` (Manager, Research)		
Priority	High		
Preconditions	`alice` has an Allocated Request with Room A, D 10:00–12:00.		
Test data	L1: Room A (named), D 10:00–12:00. L2: GPU, 4 units, D 10:00–12:00 (fits).		
Step	**Action**	**Input**	**Expected result**
1	Submit a Request B1 with L1 and L2, both Required.	L1, L2	B1 is Waiting. L1 is waitlisted with its reason; L2 is not held or allocated, although it would fit.
2	Open the GPU timeline and Research's Statement.	None	No GPU units for Research on D; no Charge and no held credits.
Pass criteria	B1 left no partial effect.		
Automated by	`RequestLifecycleTest`: "QA-atomic: when one Required line misfits, the fitting one is not held, allocated, or charged either"		

TC-LIF-03 A Request that needs an approval is Pending with Holds and held credits			
Requirements	**Lifecycle.Evaluate**, **Lifecycle.Hold.Place**, **Ledger.Account**		
Actor	`alice` (Manager, Platform)		
Priority	High		
Preconditions	Test world with the Matched data.		
Test data	M1: Person, Statistics, quantity 1, days D to D+1. L1: Room A, D 10:00–12:00 (30.00). L2: GPU, 4 units, D 10:00–12:00 (60.00).		
Step	**Action**	**Input**	**Expected result**
1	Submit a Request A1 with M1, L1, and L2.	M1, L1, L2	A1 is Pending; L1 and L2 are Held; M1 is awaiting a Candidate's selection; Alice is notified that A1 is Pending.
2	Open the Statement and Account of Platform.	None	No Charge; held credits 90.00; Balance 5000.00; Available Balance 4910.00.
3	Open Room A's timeline for D.	None	10:00–12:00 on Hold for Platform, A1.
Pass criteria	The Holds exist and nothing is charged.		
Automated by	`LedgerServiceTest`: "Ledger.Account: Balance is granted plus earned, Available Balance is Balance minus held" (credits part)		

TC-LIF-04 A Hold blocks other Requests exactly as an allocation does			
Requirements	**Lifecycle.Hold.Place**, **Metered.Pool.Check**		
Actor	`oscar` (Manager, Operations)		
Priority	High		
Preconditions	A1 as in TC-LIF-03: Room A and 4 GPU units on Hold for D 10:00–12:00.		
Test data	O1: Room A (named), D 11:00–12:00. O2: GPU, 13 units, D 10:00–12:00.		
Step	**Action**	**Input**	**Expected result**
1	Submit a Request with O1.	O1	Waiting: Room A is on Hold.
2	Submit a Request with O2.	O2	Waiting with `POOL_FULL`: $4 + 13 = 17$ units exceed 16.
Pass criteria	Neither line was allocated against the held capacity.		
Automated by	`ExclusiveAllocatorTest`: "A Hold blocks like an allocation . . . "; `MeteredAllocatorTest`: "A Hold blocks exactly like an allocation; cancel frees it"		

TC-LIF-05 Held credits stop concurrent Pending Requests from exceeding the Balance			
Requirements	**Lifecycle.Hold.Place**, **Draft.Quote.Line**		
Actor	`bob` (Manager, Research)		
Priority	High		
Preconditions	Test world with the Matched data: Research's Available Balance is 600.00.		
Test data	B1: M1 Person, Java, quantity 1, days D to D+1, and L1 Microscope 1, D 09:00–17:00 (480.00). B2: L2 Microscope 1, D+1 09:00–12:00 (180.00).		
Step	**Action**	**Input**	**Expected result**
1	Submit B1.	B1	B1 is Pending; 480.00 held; Available Balance 120.00.
2	Create B2 and submit it.	B2	The Draft shows a shortfall of 60.00; submission is refused with `INSUFFICIENT_BALANCE`.
Pass criteria	The Research Account never commits more than its Balance.		
Automated by	`LedgerServiceTest`: "Two concurrent Holds on one Account cannot together exceed the Available Balance", "INSUFFICIENT_BALANCE: a Hold above the Available Balance is refused and holds nothing"		

TC-LIF-06 The Objection Window starts at submission and lasts two working days			
Requirements	**Lifecycle.Window.Start**		
Actor	`alice` (Manager, Platform)		
Priority	High		
Preconditions	Test world with the Matched data. The case is run on a Thursday.		
Test data	A1 as in TC-LIF-03, submitted on the Thursday at time t.		
Step	**Action**	**Input**	**Expected result**
1	Submit A1 and open it.	A1	A1 is Pending and shows its Objection Window ending on the following Monday at t: two working days, skipping Saturday and Sunday.
2	Select Dave Kim; have `oscar` refuse; select Rita Gomez (TC-MAT-06).	None	A1 re-enters Pending; the window still ends on Monday at t.
Pass criteria	The window did not restart.		
Automated by	None yet		

TC-LIF-07 On lapse, every Hold is released and no Hold is placed afterwards			
Requirements	**Lifecycle.Window.Start**, **Lifecycle.Hold.Place**		
Actor	`alice` (Manager, Platform); `oscar` where named		
Priority	High		
Preconditions	A1 as in TC-LIF-03, with Dave Kim selected and Oscar silent.		
Test data	O1: Room A (named), D 10:00–12:00, for `oscar` after the lapse.		
Step	**Action**	**Input**	**Expected result**
1	[clock] Let the Objection Window lapse.	Two working days	Within five minutes: A1 is Waiting, its Holds are released, held credits 0.00, Available Balance 5000.00, and A1 is escalated.
2	As `oscar`, submit a Request with O1.	O1	Allocated: the released Hold no longer blocks Room A.
3	Open A1.	None	A1 is Waiting and holds nothing; Room A's timeline shows O1 only. No Hold was placed for A1 after the lapse.
Pass criteria	No Hold of A1 outlived its Objection Window.		
Automated by	None yet		

TC-LIF-08 When the last approval arrives, everything commits in one step			
Requirements	**Lifecycle.Allocate**, **Matched.Loan.Form**, **Draft.Submit.Lock**		
Actor	`oscar` (Manager, Operations); `alice` where named		
Priority	High		
Preconditions	A1 as in TC-LIF-03, with Dave Kim selected.		
Test data	None.		
Step	**Action**	**Input**	**Expected result**
1	Approve Dave Kim.	None	A1 is Allocated.
2	As `alice`, open A1 and the Statement.	None	L1 and L2 Allocated; Charges 30.00 and 60.00; held credits 0.00; Balance 4910.00; Alice notified.
3	Open the Statement of Operations.	None	An Earning of 800.00; Dave's Loan to Platform for D to D+1 exists.
Pass criteria	The Holds, Charges, Loan, Earning, and state change all appear together; none is missing.		
Automated by	None yet (the Exclusive and Metered part: `RequestLifecycleTest`)		

TC-LIF-09 At allocation, Optional lines that cannot be met are dropped with their reason			
Requirements	**Lifecycle.Allocate**, **Draft.Line.Required**		
Actor	`bob` (Manager, Research)		
Priority	High		
Preconditions	`alice` has an Allocated Request with Room A, D 10:00–12:00. Research's Available Balance is 600.00.		
Test data	L1: Microscope 1, D 09:00–17:00 (480.00), Required. L2: Room A (named), D 10:00–12:00, Optional (unsatisfiable). L3: Microscope 1, D+1 09:00–12:00 (180.00), Optional (not affordable after L1).		
Step	**Action**	**Input**	**Expected result**
1	Submit a Request B1 with L1, L2, and L3.	L1, L2, L3	B1 is Allocated with L1 only.
2	Open B1 and the notifications.	None	L2 Dropped, reason "no free instance"; L3 Dropped, reason "not affordable"; neither is waitlisted; Bob is told which lines were dropped and why.
3	Open the Statement of Research.	None	One Charge of 480.00.
Pass criteria	Dropped lines have no Hold, waitlist entry, or Charge.		
Automated by	`RequestLifecycleTest`: "Draft.Line.Required: an Optional line that does not fit is Dropped with its reason; the Request is still Allocated"		

TC-LIF-10 An Optional line's approval never holds a Request in Pending			
Requirements	**Lifecycle.Evaluate**, **Lifecycle.Allocate**, **Lifecycle.Window.Start**, **Matched.Escalate**		
Actor	`bob` (Manager, Research); `alice` and `erin` where named		
Priority	High		
Preconditions	Test world with the Matched data.		
Test data	L1: Room A, D 10:00–12:00 (Q1 = 30.00), Required. M6: Person, named Person Ben Ortiz, quantity 1, days D to D+1, Optional.		
Step	**Action**	**Input**	**Expected result**
1	Submit a Request B1 with L1 and M6.	L1, M6	B1 is Allocated at once; it never enters Pending. L1 is Allocated; M6 is awaiting approval, and an approval request for Ben goes to Alice. B1 shows its Objection Window ending two working days after submission.
2	Open Room A's timeline for D and the Statement of Research.	None	10:00–12:00 allocated to Research, B1, not on Hold. One Charge of 30.00; held credits 0.00.
3	[clock] Let the Objection Window lapse with Alice silent.	Two working days	Within five minutes M6's approval is escalated and Erin is notified. B1 stays Allocated and L1 keeps its allocation.
4	As `erin`, approve Ben from Escalations.	Approve	Ben's Loan to Research for D to D+1 forms now; Platform earns 800.00; Ben and Bob are notified.
Pass criteria	L1 was never held or delayed, and M6 was neither dropped at allocation nor lost at the lapse.		
Automated by	None yet		

TC-LIF-11 A Waiting Request is allocated only when every Required line fits at once			
Requirements	**Lifecycle.Waiting.Reevaluate**, **Exclusive.Waitlist.Join**		
Actor	`bob` (Manager, Research); `alice` and `oscar` where named		
Priority	High		
Preconditions	Allocated: `alice`'s A1 with Room A, D 10:00–12:00; `oscar`'s O1 with GPU, 16 units, D 10:00–12:00. Waiting: `bob`'s B1 with L1 Room A (named), D 10:00–12:00 (position 1) and L2 GPU, 6 units, D 10:00–12:00, both Required; then `oscar`'s O2 with Room A (named), D 10:00–12:00 (position 2).		
Test data	None.		
Step	**Action**	**Input**	**Expected result**
1	As `alice`, Release A1's line.	None	Room A frees. B1 cannot have L2, so B1 stays Waiting; the room passes to O2, which is Allocated.
2	Open B1.	None	B1 Waiting; L1 still at position 1 for Room A; L2 still waiting.
3	As `oscar`, Release O1's line and then O2's line.	None	After O1: L2 would fit, but Room A is O2's, so B1 waits. After O2: both fit, and B1 is Allocated at its locked Quotes (30.00 and 90.00).
Pass criteria	B1 was never partly allocated.		
Automated by	None yet		

TC-LIF-13 Two Managers submitting for the last room at once: exactly one is allocated			
Requirements	**Lifecycle.Evaluate**, **Exclusive.Conflict**		
Actor	`alice` and `bob` (Managers), at the same moment		
Priority	High		
Preconditions	Room B is allocated for D 10:00–12:00, so Room A is the last free Meeting room.		
Test data	Each submits a Request with Meeting room, quantity 1, D 10:00–12:00.		
Step	**Action**	**Input**	**Expected result**
1	Submit both Requests at the same moment.	Both Requests	Exactly one is Allocated on Room A; the other is Waiting at position 1.
2	Open both Statements.	None	One Charge of 30.00 in total; each Account equals the sum of its Transactions.
Pass criteria	Room A is never allocated twice.		
Automated by	`SubmitConcurrencyTest`: "exactly one Request is Allocated, the other Waiting at position 1, and both Accounts fold"; `AllocationConcurrencyTest`		

In the full edition only: **TC-LIF-12** Only the transitions of the state machine are allowed, and each is recorded.

4.9 Release and Refund

These cases establish that a Manager can give back an allocated Exclusive or Metered line, in whole or in part, for a Refund set by the Refund Schedule and returned to the part of the Account it came from; and that ending a Loan early corrects the lending Team's Earning, the only Transaction that may take a Balance below zero.

TC-REL-01 Release a whole line seven or more days ahead for a 75% Refund			
Requirements	**Release.Line**, **Release.Refund**		
Actor	`alice` (Manager, Platform); `bob` where named		
Priority	High		
Preconditions	`alice` has an Allocated Request A1 with L1: Room A, D 10:00–12:00, charged 30.00 (Balance 4970.00).		
Test data	Lead time from now to D 10:00: at least seven days.		
Step	**Action**	**Input**	**Expected result**
1	Open A1 and Release L1 in whole.	L1	L1 is Released; the Release is recorded with the released part, the lead time, the fraction 75%, the Refund, the actor, and the timestamp.
2	Open the Statement.	None	One Refund of 22.50 (30.00×0.75) referencing L1's Charge; Balance 4992.50.
3	Open Room A's timeline for D.	None	10:00–12:00 is free.
4	As `bob`, call the API to Release L1.	L1's id	Refused with `NOT_FOUND`: only the owning Team's Manager may Release.
Pass criteria	One Refund of 22.50 and Room A free.		
Automated by	`PricingTest`: "Release.Refund: the step function" (fraction only)		

TC-REL-02	**Partial Release shortens the window, reduces units, or removes occurrences**		
Requirements	**Release.Line**, **Release.Refund**		
Actor	`alice` (Manager, Platform)		
Priority	High		
Preconditions	`alice` has an Allocated Request A1 with: L1 GPU, 4 units, D 10:00–14:00 (charged 120.00); L2 Meeting room, weekly on D, D+7, D+14 10:00–12:00 on Room B (charged 90.00).		
Test data	All released parts start seven or more days ahead (75%).		
Step	**Action**	**Input**	**Expected result**
1	Release L1's hours 12:00–14:00.	New window D 10:00–12:00	L1 keeps 4 units for D 10:00–12:00 at its locked prices. Refund 45.00 (60.00 attributable × 0.75).
2	Release 2 of L1's units.	New units 2	L1 keeps 2 units for D 10:00–12:00. Refund 22.50 (30.00 × 0.75).
3	Release L2's occurrence on D+14.	Occurrence D+14	L2 keeps D and D+7 on Room B. Refund 22.50.
4	Open the GPU and Room B timelines.	None	GPU: 2 units for Platform on D 10:00–12:00. Room B: Platform on D and D+7 only.
Pass criteria	Three Refunds totaling 90.00; every remaining part kept its instance and prices.		
Automated by	None yet		

TC-REL-03	**The Refund fraction follows the lead time: 25% under seven days, 0% once started**		
Requirements	**Release.Refund**		
Actor	`alice` (Manager, Platform)		
Priority	High		
Preconditions	`alice` has Allocated Requests with L1: Room A, E 10:00–12:00 (charged 30.00), and L2: Room B, from H for 3 hours (charged c, its Quote).		
Test data	None.		
Step	**Action**	**Input**	**Expected result**
1	Release L1 in whole.	L1	Refund 7.50 (30.00 × 0.25): the lead time is under seven days.
2	[clock] After H, Release L2's remaining hours.	Hours from H+1	The released hours free on Room B. No credits return: the window has started, so the fraction is 0%.
Pass criteria	The Statement shows a Refund of 7.50 and no other credit for L2.		
Automated by	`PricingTest`: "Release.Refund: the step function"		

TC-REL-05	Ending a Loan early corrects the lending Team's Earning		
Requirements	**Release.Loan.Prorate**, **Ledger.Transaction.Kinds**		
Actor	`bob` (Manager, Research); `oscar` where named		
Priority	High		
Preconditions	`bob` has an Allocated Request with a Loan of Dave Kim for days D to D+2 (Wednesday to Friday); Operations received an Earning of 1200.00.		
Test data	New last day: D.		
Step	**Action**	**Input**	**Expected result**
1	End the Loan early.	Last day D	The Loan covers D only. Oscar is notified.
2	As `oscar`, open the Statement.	None	An Earning correction of −800.00 (2 person-days no longer lent) referencing the original Earning; net Earning for the Loan 400.00. The original Earning is unchanged.
3	As `bob`, open the Statement.	None	No Refund or Charge was posted to Research for the Loan.
Pass criteria	Operations' Earning for the Loan equals 400.00, the Day Rate times one day lent.		
Automated by	None yet		

In the full edition only: **TC-REL-04** A Refund returns to the granted and earned parts in the Charge's proportions; **TC-REL-06** An Earning correction may take a Balance below zero, and then blocks Exclusive and Metered Requests.

4.10 Ledger and Pricing

These cases establish that each Team's Account and Statement add up from its Transactions; that Transactions are of four kinds and never change; that Grants post and expire at the Period boundaries; that Quotes follow the price formula and its bands; and that Earnings count only working days.

TC-LED-01	Balance, held credits, and Available Balance		
Requirements	**Ledger.Account**, **Ledger.Grant.Period**		
Actor	`alice` (Manager, Platform)		
Priority	High		
Preconditions	Test world with the Matched data. `alice` has an Allocated Request A1 with Room A, D 10:00–12:00 (charged 30.00), and a Pending Request A2 with M1 (Statistics, days D to D+1, Dave Kim selected) and GPU, 4 units, D 10:00–12:00 on Hold (60.00).		
Test data	None.		
Step	**Action**	**Input**	**Expected result**
1	Open the Statement for the current Period.	None	Granted 4970.00 (5000.00 less the Charge, drawn from granted credits first), earned 0.00, held 60.00; Balance 4970.00; Available Balance 4910.00. A2 has no Charge yet.
2	Have `oscar` approve Dave Kim for A2.	None	A2 is Allocated; a Charge of 60.00 posts; held 0.00; Balance and Available Balance 4910.00.
Pass criteria	Balance = granted + earned and Available Balance = Balance − held after each step.		
Automated by	`LedgerServiceTest`: "Ledger.Account: …", "The account row equals the fold of the Team's Transactions and Period closes"		

TC-LED-02	The Statement lists the Period's Transactions, and each Team sees only its own		
Requirements	**Ledger.Account**, **Ledger.Transaction.Kinds**		
Actor	`alice` (Manager, Platform); `bob`, `ben`, and `erin` where named		
Priority	High		
Preconditions	`alice` has an Allocated Request A1 with Room A, D 10:00–12:00 (30.00) and has Released nothing.		
Test data	The current Period's id and Platform's Team id.		
Step	**Action**	**Input**	**Expected result**
1	Open Statement.	None	Opening granted 0.00 and earned 0.00; a Grant of 5000.00 and a Charge of 30.00, each with its Team, amount, Period, timestamp, actor or job, and (for the Charge) its Request Line; expired 0.00; closing granted 4970.00, earned 0.00.
2	As `bob`, call the API for Platform's Statement.	Platform's Team id	`NOT_FOUND`.
3	As `ben`, call the API for Platform's Statement.	Platform's Team id	`FORBIDDEN`: a Member who is not a Manager or the Capacity Planner sees no Statement.
4	As `erin`, open the Statement of each Team.	Platform, Research, Operations	All three are shown.
Pass criteria	The Statement's closing balances equal its opening balances plus its Transactions.		
Automated by	`LedgerControllerTest`: "Ledger.Statement: opening, totals, closing, the Account, and every Transaction of the Period", "A Manager asking for another Team's Statement gets 404 NOT_FOUND", "A Member who is neither Manager nor Planner gets 403 FORBIDDEN . . . "		

TC-LED-03	Transactions are never updated or deleted; a correction is a new Transaction		
Requirements	**Ledger.Transaction.Kinds**		
Actor	`erin` (Capacity Planner)		
Priority	High		
Preconditions	The state after TC-REL-01: Platform has a Charge of 30.00 and a Refund of 22.50.		
Test data	None.		
Step	**Action**	**Input**	**Expected result**
1	Open Platform's Statement.	None	Every Transaction is a Grant, Charge, Earning, or Refund; the Refund references the Charge it corrects.
2	Look for an edit or delete action on any Transaction, in the client and in the API document at `/api/docs/ui`.	None	None exists.
3	Run the automated test that updates and deletes a Transaction row as the application's database role.	`mvn verify`	Both statements are refused by the database.
Pass criteria	The Charge still reads 30.00 and the Refund 22.50.		
Automated by	`AppRoleGrantsTest`: "Ledger.Transaction.Kinds, QA-audit: the application role cannot change history"		

TC-LED-05	**At the Period boundary, granted credits expire, earned credits stay, and the new Grant posts**		
Requirements	**Ledger.Grant.Period**		
Actor	`oscar` (Manager, Operations); `erin` where named		
Priority	High		
Preconditions	Operations has granted 970.00 (after a Charge of 30.00) and earned 800.00 (Dave Kim's Loan, TC-MAT-10). Erin has set Operations' Grant for the next Period to 1000.00.		
Test data	None.		
Step	**Action**	**Input**	**Expected result**
1	[clock] Let the current Period end.	Period boundary	Within fifteen minutes the rollover job has run.
2	Open the Statement of the ended Period.	None	Expired 970.00; closing granted 0.00, earned 800.00.
3	Open the Statement of the new Period.	None	Opening granted 0.00, earned 800.00; a Grant of 1000.00 posted by the job; Balance 1800.00.
Pass criteria	Earned credits survived the boundary and only granted credits expired.		
Automated by	None yet		

TC-LED-06	**Quotes follow the price formula with its bands and the priority premium**		
Requirements	**Ledger.Price.Formula**		
Actor	`alice` (Manager, Platform)		
Priority	High		
Preconditions	`alice` has an empty Draft.		
Test data	The lines of each step; the expected Quotes are those of Table 4.		
Step	**Action**	**Input**	**Expected result**
1	Add a line inside the weekday band.	Meeting room, 1, D 10:00–12:00	30.00 (Q1).
2	Add a line across the band's end.	Meeting room, 1, D 16:00–18:00	25.00 (Q2).
3	Add a line outside any band on a weekday evening.	Meeting room, 1, D 18:00–20:00	20.00 ($2 \times 10.00 \times 1.0$).
4	Add a weekend line.	Meeting room, 1, D+3 10:00–12:00	10.00 (Q3).
5	Add a Priority line.	Meeting room, 1, D 10:00–12:00, Priority	60.00 (Q4).
6	Add a weekly line.	Meeting room, 1, D 10:00–12:00 on D, D+7, D+14	90.00 (Q5).
7	Add a Metered line across the band's end.	GPU, 4 units, D 16:00–18:00	50.00 (Q8).
Pass criteria	Every Quote equals the stated value to the cent.		
Automated by	`PricingTest`: "Ledger.Price.Formula: the demo story's Quotes", "Ledger.Price.Band: which band an hour falls in", "Ledger.Price.Premium and rounding"		

TC-LED-07 An Earning counts only Monday-to-Friday days of the Loan			
Requirements	Ledger.Earning.Post, Matched.Loan.Form		
Actor	`bob` (Manager, Research); `oscar` where named		
Priority	High		
Preconditions	Test world with the Matched data.		
Test data	M1: Person, skill Statistics, quantity 1, days D+2 (Friday) to D+5 (Monday).		
Step	**Action**	**Input**	**Expected result**
1	Submit a Request with M1 and select Dave Kim; as `oscar`, approve.	M1, Dave Kim	Allocated; a Loan for D+2 to D+5.
2	As `oscar`, open the Statement.	None	An Earning of 800.00: 2 person-days (Friday and Monday) × 400.00; Saturday and Sunday do not count.
Pass criteria	The Earning is 800.00, not 1600.00.		
Automated by	None yet		

TC-LED-08 [Stretch] A demand surcharge raises the Quote with the share of capacity taken and is locked			
Requirements	Ledger.Surcharge		
Actor	`erin` (Capacity Planner); `alice` and `oscar` where named		
Priority	Low		
Preconditions	Test world.		
Test data	L1: GPU, 4 units, D 10:00–12:00. O1: GPU, 8 units, D 10:00–12:00, for `oscar`.		
Step	**Action**	**Input**	**Expected result**
1	Enable the demand surcharge for GPU.	Enable	Saved and recorded.
2	As `alice`, add L1 to a Draft.	L1	Quote 60.00: nothing is allocated or held in the window, so no surcharge applies.
3	As `oscar`, submit O1. As `alice`, reopen the Draft.	O1	L1's Quote is now above 60.00: half of the pool is taken in the window.
4	As `alice`, submit, then have `oscar` Release O1.	None	L1's locked Quote and Charge do not change.
Pass criteria	The surcharge depends on the share taken at quote time and is fixed once locked.		
Automated by	None yet		

In the full edition only: **TC-LED-04** The Capacity Planner posts a mid-Period Grant with a reason.

4.11 Showback Dashboard

These cases establish that the dashboard reports utilization, idle capacity, and waiting lines per Resource Type, and credits and Loans per Team, computed from the allocations and Transactions completed before the page was requested, and that each role sees only its scope.

TC-DSH-01	Utilization, idle capacity, and waiting lines per Resource Type		
Requirements	**Dashboard.Utilization**, **Release.Line**		
Actor	`erin` (Capacity Planner)		
Priority	High		
Preconditions	E falls in the current Period. Allocated: `alice`'s Room A, E 10:00–14:00 (4 instance-hours) and GPU, 4 units, E 10:00–12:00 (8 unit-hours). Waiting: `bob`'s Room A (named), E 11:00–12:00. Let h be the number of hours in the current Period.		
Test data	None.		
Step	**Action**	**Input**	**Expected result**
1	Open the dashboard for the current Period.	None	Meeting room: utilization $4 \div (2 \times h) \times 100\%$ (for a 30-day Period, $h = 720$: 0.28%); 1 line waiting. GPU: utilization $8 \div (16 \times h) \times 100\%$ (0.07% for $h = 720$); 0 lines waiting. Confocal microscope: 0%.
2	Read idle capacity for the coming seven days.	None	Meeting room $2 \times 168 - 4 = 332$ instance-hours; GPU $16 \times 168 - 8 = 2680$ unit-hours; Confocal microscope 168.
3	Have `alice` Release the GPU line, then reload the dashboard.	None	GPU utilization 0% and idle capacity 2688 unit-hours: the Release completed before the page was requested.
Pass criteria	Every figure matches the formula to the stated rounding; the room allocated but not Released stays counted as allocated.		
Automated by	None yet		

TC-DSH-02	Credits and Loans per Team and Period		
Requirements	**Dashboard.Credits**		
Actor	`erin` (Capacity Planner)		
Priority	High		
Preconditions	In the current Period: Platform charged 30.00 (Room A, D 10:00–12:00) and refunded 22.50 (its Release, TC-REL-01); Research borrowed Dave Kim for days E to E+1, two weekdays inside the current Period (formed as in TC-MAT-10), so Operations earned 800.00.		
Test data	None.		
Step	**Action**	**Input**	**Expected result**
1	Open the dashboard's Team figures for the current Period.	None	Platform: Grants 5000.00, Charges 30.00, Earnings 0.00, Refunds 22.50, expired 0.00, Balance and Available Balance 4992.50. Research: Grants 600.00; Charges, Earnings, Refunds, and expired 0.00; Balance and Available Balance 600.00; person-days borrowed 2. Operations: Grants 1000.00, Earnings 800.00, Balance and Available Balance 1800.00, person-days lent 2.
2	Follow Platform's link to its Statement.	None	The Statement of Platform for the Period opens and agrees with the figures.
Pass criteria	Each Team's figures equal the sums of its Transactions and Loans in the Period.		
Automated by	None yet		

TC-DSH-04	[Stretch] Requested versus used units per Team and Metered type		
Requirements	**Dashboard.Usage**, **Metered.Usage.Compare**		
Actor	`erin` (Capacity Planner)		
Priority	Low		
Preconditions	The state after TC-MET-07: Platform requested 4 GPU units for E 10:00–12:00 and 3 were recorded as used.		
Test data	The ended Period.		
Step	**Action**	**Input**	**Expected result**
1	Open the dashboard for the ended Period.	The Period	GPU, Platform: requested 8 unit-hours, used 6 unit-hours.
2	Open Platform's Statement for the Period.	None	Unchanged: the Charge is 60.00.
Pass criteria	Usage is shown and has no Ledger effect.		
Automated by	None yet		

In the full edition only: **TC-DSH-03** Each role sees the dashboard in its own scope.

4.12 Administration

These cases establish that the Administrator manages Members, Teams, skills, and roles but no policy; that sign-in and roles are enforced by the server; and that the Capacity Planner sets every policy value as data, with each change recorded and affecting only what the requirement says it affects.

TC-ADM-01	Create a Member who can sign in; a deactivated Member cannot		
Requirements	**Admin.Member.Create**, **Matched.Shortlist.Build**		
Actor	`carol` (Administrator); the new Member and `alice` where named		
Priority	High		
Preconditions	Test world with the Matched data.		
Test data	Member N1: name "Nina Lowe", email `nina@openbench.example`, username `nina`, password `start-2026`, Team Operations; skill Statistics and availability to D+60 added by Nina.		
Step	**Action**	**Input**	**Expected result**
1	Create N1 under Members.	N1	Nina is listed in Operations with the role Member only.
2	Sign in as `nina`.	`nina` / `start-2026`	Signed in; the shell shows her name and Team. She adds her skill and availability.
3	Sign in as `nina` with a wrong password.	`nina` / `wrong`	Refused with `BAD_CREDENTIALS`; the message does not say whether the username exists, and is the same for an unknown username.
4	As `alice`, submit a line for Statistics, days D to D+1, and open its Shortlist.	None	Dave Kim, Nina Lowe, Rita Gomez.
5	As `carol`, deactivate Nina; as `alice`, submit the same line again.	None	Nina cannot sign in (refused as a wrong login), and she is not on the new Shortlist; her history is kept.
Pass criteria	Nina's account works only while active.		
Automated by	`SessionTest`: "CO-auth, SE-password: session login with CSRF" (sign-in part)		

TC-ADM-02	Teams, moves between Teams, and roles keep exactly one Manager per Team		
Requirements	**Admin.Member.Create**		
Actor	`carol` (Administrator)		
Priority	High		
Preconditions	Test world.		
Test data	Team “Design”. Members Ben Ortiz and Rita Gomez.		
Step	**Action**	**Input**	**Expected result**
1	Create the Team Design, then rename it.	“Design”, then “Product Design”	Listed as Product Design, with no Members yet.
2	Move Ben to Product Design and grant him Manager.	Ben, Manager	Ben is the Manager of Product Design; Platform still has Alice as Manager. The change is recorded with actor, old and new value, and timestamp.
3	Grant Manager to Rita (Research).	Rita, Manager	Rita replaces Bob as Research’s Manager; Bob keeps the role Member only. Research has exactly one Manager.
4	Grant and revoke Resource Owner for Rita.	Rita	Granted, then revoked; both recorded.
5	Retire Product Design.	None	Retired; its history is kept.
Pass criteria	Every Team has exactly one Manager after each step.		
Automated by	None yet		

TC-ADM-04	Roles are enforced by the server, not only by the client		
Requirements	**Admin.Member.Create**		
Actor	`ben` (Member only)		
Priority	High		
Preconditions	Test world. A cookie jar with Ben’s session and CSRF token.		
Test data	The calls of each step.		
Step	**Action**	**Input**	**Expected result**
1	Define a Resource Type.	POST `/api/inventory/types`	`FORBIDDEN`; no type is added.
2	Add a Resource.	POST `/api/inventory/resources`	`FORBIDDEN`.
3	Create a Request.	POST `/api/requests`	`FORBIDDEN`.
4	Post a Grant.	POST `/api/ledger/grants`	`FORBIDDEN`.
5	Repeat step 1 without the CSRF header.	No `X-XSRF-TOKEN`	`CSRF_REJECTED`.
6	Sign out and call `/api/me`.	None	`UNAUTHENTICATED`.
Pass criteria	Nothing was created.		
Automated by	`InventorySecurityTest`: “SE-roles: a write without its role is refused with 403 FORBIDDEN and changes nothing”; `MembersControllerTest`; `SessionTest`		

TC-ADM-05	The Capacity Planner sets each policy value as data, and each change is recorded		
Requirements	**Planner.Settings**, **Admin.Member.Create**		
Actor	`erin` (Capacity Planner)		
Priority	High		
Preconditions	Test world.		
Test data	Day Rate 450.00; priority premium 3; Objection Window 3 working days; bump cutoff for Meeting room 48 hours; Quota for Platform on GPU 10; Grant for Research for the next Period 700.00; Price Band for GPU Saturday and Sunday ×0.25; Refund Schedule for Meeting room 50% at 168 hours, 10% below.		
Step	**Action**	**Input**	**Expected result**
1	Open Settings and change each value.	The test data	Each is saved without a restart and shown at once.
2	Open the settings history.	None	One entry per change with actor Erin Walsh, the old value, the new value, and the timestamp.
3	Add a Price Band overlapping an existing one.	GPU, Monday 12:00–18:00 ×2.0	Refused with `BANDS_OVERLAP` and nothing saved: Price Bands of a type may not overlap.
4	Set a Refund Schedule step of 100%.	Meeting room, 100% at 336 hours	Refused with `REFUND_STEP_TOO_HIGH` and nothing saved: no step may be 100% or more.
5	Set a negative Day Rate.	−1.00	Refused with `VALUE_OUT_OF_RANGE` and nothing saved; a Day Rate of 0.00 is allowed.
Pass criteria	Only valid values were saved, and every saved change is in the history.		
Automated by	None yet		

TC-ADM-06	A changed setting affects only what comes after the change		
Requirements	**Planner.Settings**, **Release.Refund**		
Actor	`erin` (Capacity Planner); `alice` where named		
Priority	High		
Preconditions	`alice` has an Allocated Request A1 with Room A, D 10:00–12:00 (locked 30.00), and a Draft A2 with Room B, D 10:00–12:00.		
Test data	Refund Schedule for Meeting room: 50% at 168 hours, 10% below. Platform's planned Grant: 6000.00.		
Step	**Action**	**Input**	**Expected result**
1	Change the Refund Schedule.	The new schedule	Saved.
2	As `alice`, Release A1's line.	None	Refund 15.00 (30.00 × 0.50): the Release happens after the change.
3	Change Platform's planned Grant, first for the current Period, then from the next Period.	6000.00	For the current Period: refused with `PERIOD_STARTED`. From the next Period: saved; the current Period's posted Grant of 5000.00 is unchanged (a mid-Period Grant would be needed to add credits now).
4	Change the Unit Price of Meeting room to 20.00 and open A2.	20.00	A2's Quote is 60.00; A1's locked Quote is still 30.00.
Pass criteria	No posted Transaction or locked Quote changed.		
Automated by	`RequestLifecycleTest`: "Draft.Submit.Lock: …" (price part)		

In the full edition only: **TC-ADM-03** The Administrator maintains the skill list and has no policy settings.

A Traceability Matrix

Every functional requirement of the SRS short edition, version 0.4, in the order of its Section 4, with the test cases of this edition that verify it. The matrix is generated from the requirement list of each test case; the full edition's matrix adds the Medium priority cases.

41 of the 41 Release 1 requirements and 47 of the 47 requirements in scope are covered by at least one test case; the document has 68 test cases.

Requirement	Scope	Test cases
Inventory and Resource Types		
Inventory.Type.Define	Release 1	TC-INV-01
Inventory.Resource.Add	Release 1	TC-INV-04, TC-INV-05, TC-INV-06
Inventory.View.Holder	Release 1	TC-INV-08
Member Profiles		
Profile.Skills	Release 1	TC-PRF-01
Profile.Availability	Release 1	TC-PRF-02, TC-PRF-03
Suggestions		
Suggest.Create	Release 1	TC-SUG-01
Suggest.Accept.Existing	Release 1	TC-SUG-02
Draft and Submit a Request		
Draft.Create	Release 1	TC-DRF-01, TC-DRF-02, TC-DRF-10
Draft.Line.Add	Release 1	TC-DRF-03, TC-DRF-05
Draft.Line.Required	Release 1	TC-DRF-06, TC-LIF-09
Draft.Quote.Line	Release 1	TC-DRF-03, TC-DRF-06, TC-DRF-07, TC-LIF-05
Draft.Submit.Lock	Release 1	TC-DRF-09, TC-LIF-08
Matched Allocation		
Matched.Shortlist.Build	Release 1	TC-PRF-02, TC-PRF-03, TC-MAT-01, TC-MAT-03, TC-ADM-01
Matched.Approve.Route	Release 1	TC-MAT-03, TC-MAT-04
Matched.Refuse.Replace	Release 1	TC-MAT-06, TC-MAT-07
Matched.Escalate	Release 1	TC-MAT-08, TC-LIF-10
Matched.Loan.Form	Release 1	TC-MAT-10, TC-LIF-08, TC-LED-07
Exclusive Allocation		
Exclusive.Conflict	Release 1	TC-EXC-01, TC-LIF-13
Exclusive.Waitlist.Join	Release 1	TC-EXC-04, TC-EXC-05, TC-LIF-11
Exclusive.Priority.Bump	Release 1	TC-EXC-07, TC-EXC-09
Exclusive.Bump.Effect	Release 1	TC-EXC-08
Metered Allocation		
Metered.Line.Form	Release 1	TC-MET-01
Metered.Pool.Check	Release 1	TC-MET-02, TC-MET-03, TC-LIF-04
Metered.Tier.Define	Stretch	TC-MET-05
Metered.Tier.Bump	Stretch	TC-MET-06
Metered.Usage.Record	Stretch	TC-MET-07
Metered.Usage.Compare	Stretch	TC-MET-07, TC-DSH-04
Request Lifecycle and Holds		
Lifecycle.States	Release 1	TC-DRF-01, TC-MAT-07, TC-EXC-08, TC-LIF-01, TC-LIF-02
Lifecycle.Evaluate	Release 1	TC-MAT-01, TC-MAT-07, TC-EXC-01, TC-LIF-01, TC-LIF-02, TC-LIF-03, TC-LIF-10, TC-LIF-13
Lifecycle.Hold.Place	Release 1	TC-INV-08, TC-DRF-10, TC-LIF-03, TC-LIF-04, TC-LIF-05, TC-LIF-07
Lifecycle.Window.Start	Release 1	TC-MAT-06, TC-MAT-08, TC-LIF-06, TC-LIF-07, TC-LIF-10
Lifecycle.Allocate	Release 1	TC-LIF-01, TC-LIF-08, TC-LIF-09, TC-LIF-10
Lifecycle.Waiting.Reevaluate	Release 1	TC-MAT-06, TC-EXC-05, TC-LIF-11
Release and Refund		
Release.Line	Release 1	TC-EXC-05, TC-REL-01, TC-REL-02, TC-DSH-01
Release.Refund	Release 1	TC-REL-01, TC-REL-02, TC-REL-03, TC-ADM-06
Release.Loan.Prorate	Release 1	TC-PRF-03, TC-REL-05
Ledger and Pricing		
Ledger.Account	Release 1	TC-DRF-07, TC-LIF-03, TC-LED-01, TC-LED-02
Ledger.Transaction.Kinds	Release 1	TC-REL-05, TC-LED-02, TC-LED-03
Ledger.Grant.Period	Release 1	TC-LED-01, TC-LED-05

Requirement	Scope	Test cases
Ledger.Price.Formula	Release 1	TC-DRF-03, TC-EXC-07, TC-MET-01, TC-LED-06
Ledger.Earning.Post	Release 1	TC-MAT-10, TC-LED-07
Ledger.Surcharge	Stretch	TC-LED-08
Showback Dashboard		
Dashboard.Utilization	Release 1	TC-DSH-01
Dashboard.Credits	Release 1	TC-DSH-02
Dashboard.Usage	Stretch	TC-DSH-04
Administration		
Admin.Member.Create	Release 1	TC-ADM-01, TC-ADM-02, TC-ADM-04, TC-ADM-05
Planner.Settings	Release 1	TC-DRF-09, TC-MAT-08, TC-ADM-05, TC-ADM-06

B Glossary

The terms used in the test cases, with the meanings the SRS gives them; generated from the project's shared vocabulary.

People and roles

Member Any user of the system. Belongs to exactly one Team, has a skill profile and availability, and is lendable as a Person.

Team The organizational unit that owns an Account and every Request made on its behalf. Has exactly one Manager.

Manager The Member who leads a Team. The only actor who submits Requests, and the approver for lending that Team's Members.

Resource Owner A Member responsible for the Inventory records of specific Exclusive or Metered resources. Never approves allocations.

Capacity Planner The role that sets each Team's Grant per Period, Unit Prices, the Day Rate, and Refund Schedules, and receives escalations.

Administrator The role that manages users, Teams, and Resource Type definitions. Configures the system; sets no policy.

Resources

Resource Anything that can be allocated: a Person, an Exclusive resource such as a room, or a Metered pool such as GPU hours. Every Resource has exactly one Resource Type.

Resource Type A named definition, created as data by an Administrator, that fixes a Resource's Allocation Class and attributes.

Allocation Class One of three policies a Resource Type declares, deciding how Requests for it are allocated: Matched, Exclusive, or Metered.

Matched The Allocation Class for People. Allocated by matching skills and availability, gated by the Person's Manager.

Person A Member viewed as a Matched Resource: someone with skills and availability who can be lent to another Team.

Exclusive The Allocation Class for Resources held whole over a time window, such as rooms or instruments. Allocated with conflict detection, a waitlist, and priority bumping.

Metered The Allocation Class for divisible Resources drawn against a Team quota, such as compute hours or license seats.

Pool Capacity The total units a Metered Resource Type can supply at once, such as 32 GPUs, maintained by its Resource Owner. Allocations in any hour may not exceed it.

Quota An optional per-Team cap on the units of one Metered Resource Type the Team may hold concurrently, set by the Capacity Planner. A safety valve against monopolizing a pool, not a rationing mechanism.

Inventory The record of every Resource that exists, its Resource Type, and who currently holds it or has it on Hold.

Requests

Request A single request, owned by a Team and submitted by its Manager, containing one or more Request Lines that may span all three Allocation Classes.

Request Line One need inside a Request: a Resource Type, a quantity, and a time window. Every line has exactly one Allocation Class, taken from its Resource Type.

Suggestion A single Request Line a Member proposes to their own Manager. Has no allocation effect until the Manager accepts it into a Request; several Suggestions may be accepted into the same line.

Required line / Optional line A Manager's marking on each Request Line. A Request is fulfilled only when every Required line is allocated; Optional lines allocate if they can and never block the outcome.

Hold A tentative, uncharged allocation of an Exclusive or Metered line placed while a Request's Matched lines await approval. Commits when the Request is fulfilled, releases when it fails or the Objection Window lapses.

Objection Window The bounded period a lending Manager has to approve or refuse a Matched line before the Request escalates to the Capacity Planner. Also the lifetime of any Hold on that Request.

Candidate A Person who satisfies a Matched line's skills and availability, has no Loan overlapping the line's window, and appears on its Shortlist. Becomes a Loan once their Manager approves.

Shortlist The list of Candidates the system produces for a skill-based Matched line, ordered by fewest person-days lent in the current Period, from which the requesting Manager selects.

Waitlist The ordered list of Required lines that could not be satisfied for a Resource Type and window. Priority lines first, then by submission time; a Bumped line goes to the head. Optional lines are never waitlisted.

Loan A Person allocated to a Team other than their own for a time window. The fulfilled form of a Matched line. A Person is in at most one Loan at a time.

Priority line An Exclusive Request Line that is charged a premium, a multiple of the Unit Price set by the Capacity Planner. Can Bump a standard line and cannot itself be Bumped.

Bump The displacement of a standard Exclusive line by a Priority line. The displaced line goes to the head of the waitlist with a full Refund. Not allowed inside the bump cutoff before the window starts.

Release A Manager giving back an allocated Exclusive or Metered line, in whole or in part, before it is consumed, or ending a Loan early. Triggers a Refund according to the Refund Schedule.

Credits

Credit The unit of account in the Ledger. No real money; used for showback, not billing.

Ledger The record of every Team's Account and every Transaction. Holds no capacity information.

Account A Team's standing in the Ledger: its granted credits, its earned credits, and its held credits.

Balance A Team's granted credits plus earned credits.

Available Balance Balance minus held credits. What a Team can commit to a new Request.

Held credits Credits set aside for a Request that is Pending, equal to what its Holds will cost when they commit.

Transaction One entry in the Ledger against an Account. Exactly four kinds: a Grant (from the Capacity Planner), a Charge (for an allocated Exclusive or Metered line), an Earning (for a Loan), or a Refund (for a Release).

Grant Credits given to a Team by the Capacity Planner at the start of a Period, or as a mid-period adjustment. Expire at the end of the Period.

Period The fixed calendar interval over which Grants are issued and expire. Set by the Capacity Planner.

Statement A Team's per-Period view of its Account: Grants, Charges, Earnings, Refunds, and what expired. The showback artifact.

Unit Price The base credits per unit of an Exclusive or Metered Resource Type (per hour of a room, per GPU hour), set by the Capacity Planner, before any Price Band multiplier. Matched lines have no Unit Price.

Price Band A time-of-day and day-of-week window, defined per Resource Type by the Capacity Planner, carrying a multiplier on the Unit Price. Rewards use at low-demand times.

Quote The credits a Request Line will cost, shown to the Manager while drafting and locked when the Request is submitted. The Charge that posts equals the Quote.

Day Rate The flat credits a Team earns per person-day of a Loan, set once by the Capacity Planner. The same for every Person regardless of skill.

Refund Schedule The rule, set per Resource Type by the Capacity Planner, giving the fraction of a Charge returned on Release as a step function of how far ahead of the window start the Release happens. Never reaches 100%.